How Our Drone Knows Where It Is and How It Is Controlled

The complete mathematics of the CD2 drone,
explained step by step with toy examples

3D Mapping of an Environment using a Normal Monocular Camera
and Controlling the Drone by Visual-Inertial Odometry (VIO)

Introduction to Drones – 22AIE448

Group CD2

Ravishanmugam K ▪ Ishwarya Murugesan ▪ Aparna Bharani ▪ Cibikumar B ▪ Akhilan S

School of Artificial Intelligence, Amrita Vishwa Vidyapeetham

What this document is. One place that explains everything in our project: the maths of how a drone moves, how it estimates where it is from a camera and an IMU, how it is controlled, and how each equation appears in our MATLAB toy model (`toy_model3`) and in our ROS 2 controller (`drones_controller`). Nothing is assumed except school algebra and a little calculus. Every new idea gets a small example with numbers that you can check with a calculator.

Contents

1 Read This First: the Big Picture **2**

1.1 What our project does, in one paragraph . . . 2
1.2 The four questions a drone must answer, 25 times every second . . . 2
1.3 The whole loop in one picture . . . 2
1.4 Two ways of building the controller: Model 1 and Model 2 . . . 2
1.5 How every block is explained . . . 3

2 The Maths Toolbox (everything we need, with tiny examples) **6**

2.1 Vectors and matrices . . . 6
2.2 Rates of change and differential equations . . . 6
2.3 Taylor series and linearization . . . 7
2.4 Rotations: the rotation matrix R . . . 7
2.5 The hat and vee maps . . . 8
2.6 Exp and Log: rotation vector $\leftrightarrow$ rotation matrix . . . 8
2.7 Uncertainty: mean, variance, covariance . . . 9
2.8 Finding the lowest point of a bowl . . . 9
2.9 Eigenvalues: will it blow up or die out? . . . 9
2.10 Least squares and the pseudo-inverse . . . 10

3 Our Drone and Our Software **10**

3.1 The pieces of the system . . . 10
3.2 Frames: which way is up? . . . 10
3.3 The state of the drone . . . 11
3.4 The parameters: `controller.yaml` . . . 11

4 Model 1: Physics, Estimation and Geometric Control **12**

4.1 Block 1 — How the drone moves (nonlinear dynamics) . . . 12
4.2 Block 2 — Linearization: zooming in on the dynamics . . . 15
4.3 Block 3 — How a small error grows (local error dynamics) . . . 17
4.4 Block 4 — Where am I? The error-state Kalman filter (our VIO) . . . 19
4.5 Block 5 — Position control (outer loop) . . . 23
4.6 Block 6 — Force and attitude: which way to lean . . . 25
4.7 Block 7 — Attitude control on SO(3) (inner loop) . . . 27
4.8 Block 8 — Four motor speeds (control allocation) . . . 30
4.9 All of Model 1 together: the closed loop . . . 31

5 How Our ROS 2 Node Actually Controls the Drone 32

5.1 Start-up: reading the parameters and building the controller . . . 33

5.2 Between control steps: receiving the state (block 4's output) . . . 34

5.3 One control step, line by line . . . 35

5.4 What is actually sent to the drone: the velocity command . . . 36

5.5 Summary: from equation to line of code . . . 38

6 Model 2: Learning from Data and Optimal Control 38

6.1 Block 1 — Recording flight data . . . 39

6.2 Block 2 — DMDc: learning A and B from the data . . . 40

6.3 Block 3 — The Bellman equation: what is "the best" controller? . . . 42

6.4 Block 4 — The Linear Quadratic Regulator (LQR) . . . 43

6.5 Block 5 — The Riccati recursion: finding P step by step . . . 44

6.6 Block 6 — The DARE: jumping straight to the answer . . . 45

6.7 Block 7 — Model Predictive Control (MPC): planning with limits . . . 46

6.8 Block 8 — Turning MPC into a Quadratic Program (QP) . . . 47

6.9 Block 9 — Receding horizon: plan, act a little, plan again . . . 49

6.10 Block 10 — Applying it to the drone . . . 50

6.11 Block 11 — Working out the state: an Extended Kalman Filter . . . 52

7 How Everything Fits Together 55

7.1 One trip around the loop, in words . . . 55

7.2 Model 1 or Model 2? . . . 56

7.3 Why the camera matters for control . . . 56

8 Using the Toy Model 57

8.1 The files . . . 57

8.2 How to run them . . . 57

8.3 How each live script is organised . . . 57

8.4 Things to try . . . 58

A All Symbols in One Table 59

B Glossary 60

C References 60

1 Read This First: the Big Picture

1.1 What our project does, in one paragraph

Our drone carries an ordinary camera (one lens, a *monocular* camera) and an IMU (an Inertial Measurement Unit: a gyroscope that measures how fast the drone is turning, and an accelerometer). From the camera images we work out how the drone has moved; this is called *visual odometry*. Mixing that with the IMU is called *Visual-Inertial Odometry* (VIO). VIO tells the drone where it is, and the same camera poses are used to build a 3D map of the room. A controller then uses "where am I" to decide how to move to "where I should be". In our setup the drone is the Iris quadrotor flying in the Gazebo simulator, ArduPilot (running as SITL, *Software In The Loop*) is the flight computer, and our own ROS 2 node `drones_controller` is the controller.

1.2 The four questions a drone must answer, 25 times every second

Question	What answers it in this document	Where
1. *How does a drone move at all?*	Physics: Newton's law for moving and Euler's law for turning	Section 4.1–4.3
2. *Where am I right now?*	A Kalman filter that mixes camera and IMU with a model: our VIO (physics in Model 1, the learned model in Model 2)	Section 4.4, 6.11
3. *Where should I go, and how hard should I push?*	A controller: geometric SO(3) control (Model 1) or optimal LQR / MPC (Model 2)	Section 4.5–4.7, 6
4. *How do I make four motors do that?*	Control allocation: splitting thrust and turning force between the four rotors	Section 4.8

1.3 The whole loop in one picture

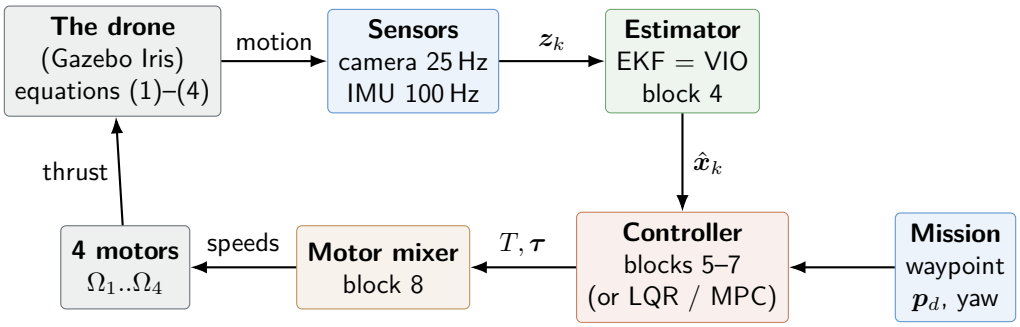


Figure 1: The closed loop. The drone moves, the sensors measure it, the estimator works out the state $\hat{x}$, the controller compares it with the mission and decides thrust T and torque τ, the mixer turns that into four motor speeds, and the drone moves again. This repeats every 40 ms (25 Hz).

1.4 Two ways of building the controller: Model 1 and Model 2

We studied the problem in two ways. Both end with the same four motor speeds.

- **Model 1, the physics way** (section 4). Write down Newton's and Euler's laws for the drone, use them to estimate the state (the EKF) and control the drone with a controller that works directly with rotation matrices. This is the structure of our ROS 2 node.
- **Model 2, the data way** (section 6). Fly, record, let the computer find a simple linear model

that explains the recording (DMDc), and then compute the best possible controller for that model (LQR), or the best plan that also respects limits such as "never faster than 1 m/s" (MPC).

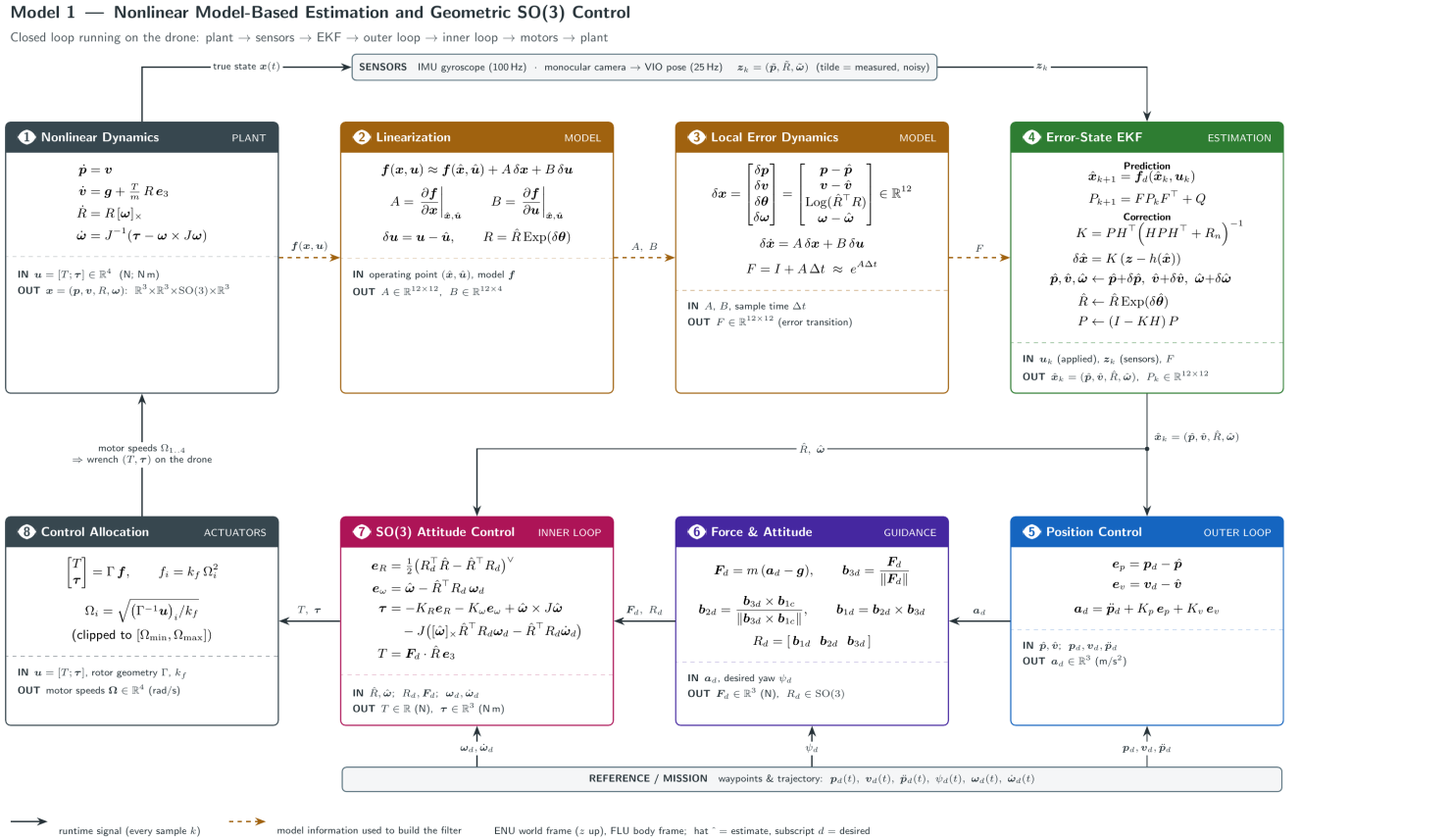
The two flow diagrams (figs. 2 and 3) show every block with its equations. Every block number in those diagrams is a subsection number in this document.

1.5 How every block is explained

Every block follows the same pattern, with the same coloured boxes:

Part	What it gives you
The idea in plain words (blue)	What the block does, with no maths
Equation box (orange)	The equations, numbered as in the flow diagram
Derivation	Where each equation comes from, one small step at a time
Toy example (brown)	Small numbers that you can check with a calculator
Toy model code (grey)	The lines of the MATLAB live script in `toy_model3` that compute the equation
How this controls the real drone (green)	The lines of our Python controller (with line numbers) and what they do on the drone

The toy model. `toy_model3` contains six MATLAB live scripts (`.mlx`) and one helper file `DroneLib.m`. Each live script mixes explanation, equations and code, like the course notes. The numbers printed in this document are the numbers those scripts print. Section 8 explains how to run them.



Notation and dimensions

States, inputs, parameters				Linearization and SO(3) tools			Estimation (EKF)				Control and actuation			
p	position (world, ENU)	$\mathbb{R}^3$	m	$[a]_\times$	skew matrix, $[a]_\times b = a \times b$	$\mathbb{R}^{3\times3}$	–	$\hat{x}$	estimated state $(\hat{p}, \hat{v}, \hat{R}, \hat{\omega})$	18 nos.	mixed	$p_d, v_d, \ddot{p}_d$	desired pos./vel./acc.	$\mathbb{R}^3$	m, m/s, m/s²
v	velocity (world)	$\mathbb{R}^3$	m/s	$(\cdot)^\vee$	inverse of $[\cdot]_\times$	$\mathbb{R}^3$	–	Δt	EKF / IMU sample time	$\mathbb{R}$	s (0.01)	e_p, e_v	position / velocity error	$\mathbb{R}^3$	m, m/s
R	attitude, body → world	$SO(3)$	–	Exp, Log	rot. vector ↔ rot. matrix	$\mathbb{R}^3 \leftrightarrow SO(3)$	rad	P_k	error covariance	$\mathbb{R}^{12\times12}$	m^2, rad^2, …	K_p, K_v	position / velocity gains	$\mathbb{R}^{3\times3}$	s^{-2}, s^{-1}
ω	body angular velocity	$\mathbb{R}^3$	rad/s	$\hat{(\cdot)}$	estimate (EKF output)	–	–	Q	process-noise covariance	$\mathbb{R}^{12\times12}$	as P	a_d	desired acceleration	$\mathbb{R}^3$	m/s²
T	total thrust (body z)	$\mathbb{R}$	N	$\bar{x}, \bar{u}$	operating point	18, $\mathbb{R}^4$	mixed	z	measurement (cam: 6, gyro: 3)	$\mathbb{R}^m$	m, rad, rad/s	F_d	desired force	$\mathbb{R}^3$	N
τ	body torque	$\mathbb{R}^3$	N m	δx	error state $[\delta p; \delta v; \delta\theta; \delta\omega]$	$\mathbb{R}^{12}$	m, m/s, rad, rad/s	$h(\cdot)$	measurement model	$\mathbb{R}^{18} \to \mathbb{R}^m$	–	$\psi_d,\ b_{1c}$	desired yaw, heading	$\mathbb{R},\ \mathbb{R}^3$	rad, –
u	input $[T; \tau]$	$\mathbb{R}^4$	N, N m	$\delta\theta$	attitude error, $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$	$\mathbb{R}^3$	rad	H	measurement Jacobian	$\mathbb{R}^{m\times12}$	–	b_{1d}, b_{2d}, b_{3d}	desired body axes (unit)	$\mathbb{R}^3$	–
x	state (p, v, R, ω)	12 DOF	mixed	δu	input error $u - \hat{u}$	$\mathbb{R}^4$	N, N m	R_n	measurement-noise covariance	$\mathbb{R}^{m\times m}$	as z^2	R_d	desired attitude	$SO(3)$	–
m	mass	$\mathbb{R}$	kg (1.5)	f	dynamics $\dot{x} = f(x, u)$	12	mixed	K	Kalman gain	$\mathbb{R}^{12\times m}$	mixed	e_R, e_ω	attitude / rate error	$\mathbb{R}^3$	–, rad/s
J	inertia $\mathrm{diag}(0.02, 0.02, 0.04)$	$\mathbb{R}^{3\times3}$	kg m²	A	state Jacobian $\partial f/\partial x$	$\mathbb{R}^{12\times12}$	mixed	f_d	one discrete step of f	$18 \to 18$	mixed	$\omega_d, \dot{\omega}_d$	desired rate, rate derivative	$\mathbb{R}^3$	rad/s, rad/s²
g	gravity $[0, 0, -9.81]^\top$	$\mathbb{R}^3$	m/s²	B	input Jacobian $\partial f/\partial u$	$\mathbb{R}^{12\times4}$	mixed	$\delta\hat{x}$	correction computed by the EKF	$\mathbb{R}^{12}$	mixed	K_R, K_ω	attitude / rate gains	$\mathbb{R}^{3\times3}$	N m, N m s
e_3	unit vector $[0, 0, 1]^\top$	$\mathbb{R}^3$	–	F	discrete error transition	$\mathbb{R}^{12\times12}$	–	I	identity matrix	$\mathbb{R}^{12\times12}$	–	$\Gamma,\ f,\ k_f$	mixer, rotor thrusts, thrust const.	$\mathbb{R}^{4\times4}, \mathbb{R}^4, \mathbb{R}$	–/m, N, N s²
$(\cdot)_k$	value at $t_k = k\,\Delta t$	–	–									Ω	rotor speeds	$\mathbb{R}^4$	rad/s

Figure 2: Model 1: nonlinear physics, error-state EKF (our VIO) and geometric SO(3) control.

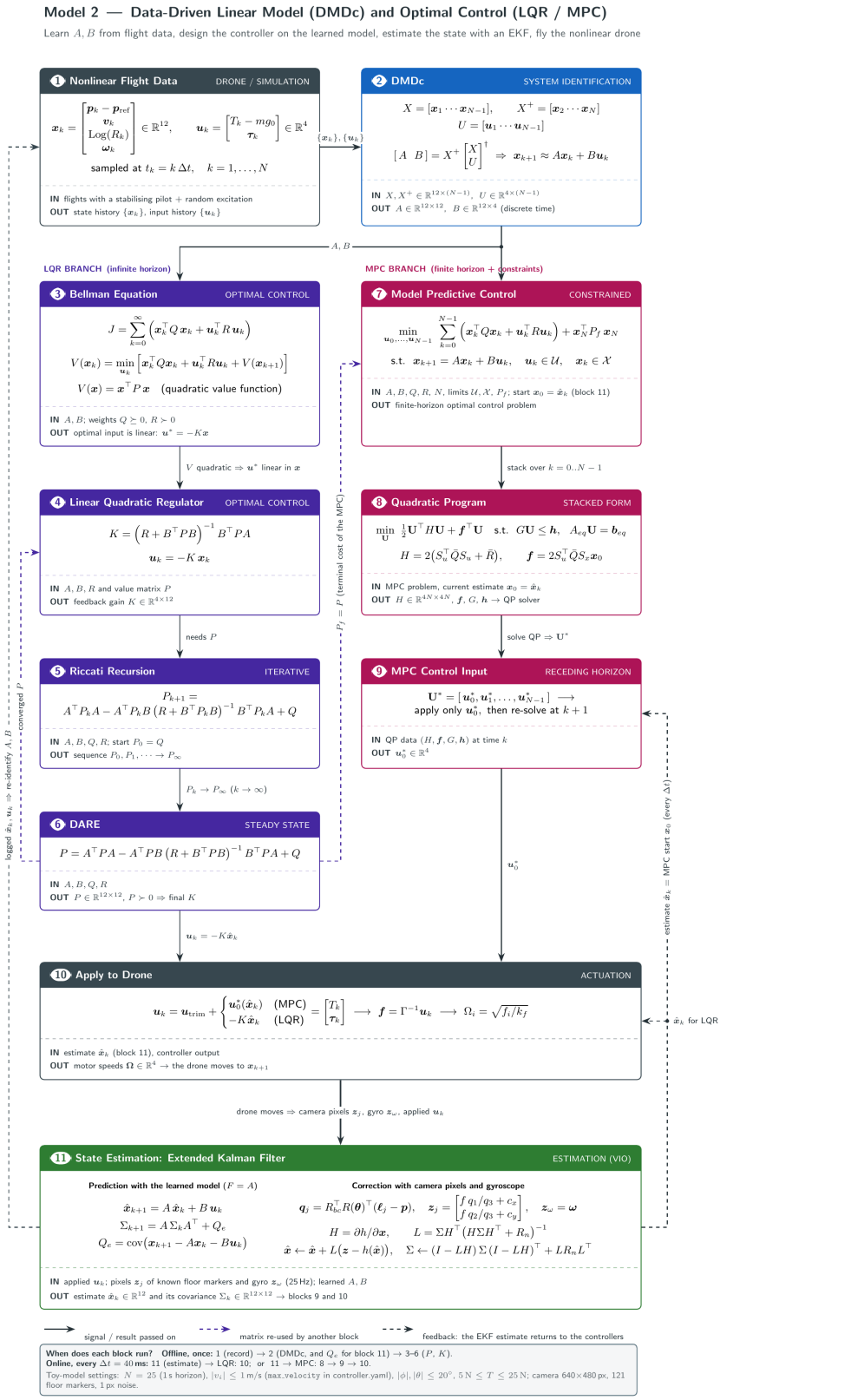


Figure 3: Model 2: a linear model learned from flight data (DMDc), then optimal control by LQR (left

2 The Maths Toolbox (everything we need, with tiny examples)

This section collects every mathematical tool used later. If a later page uses a word you do not know, it is explained here. Each tool comes with the smallest example that shows how it works.

2.1 Vectors and matrices

A **vector** is a list of numbers written as a column. The position of the drone is a vector with three numbers:

$$\boldsymbol{p} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} = \begin{bmatrix} 0.1 \\ -0.2 \\ 1.5 \end{bmatrix} \text{ m} \quad \text{means: 10 cm east, 20 cm south, 1.5 m up.}$$

We write $\boldsymbol{p} \in \mathbb{R}^3$ ("$\boldsymbol{p}$ is a list of 3 real numbers"). A **matrix** is a table of numbers. $M \in \mathbb{R}^{m \times n}$ has m rows and n columns. Multiplying a matrix by a vector takes each row, multiplies it entry by entry with the vector and adds up:

Toy example — matrix times vector

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 6 \\ 3 \cdot 5 + 4 \cdot 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}.$$

Sizes must fit: a 4×12 matrix times a 12-vector gives a 4-vector. This is exactly what the LQR controller does: 12 errors in, 4 commands out.

The **transpose** $M^\top$ swaps rows and columns. For vectors, $\boldsymbol{a}^\top \boldsymbol{b} = a_1 b_1 + a_2 b_2 + a_3 b_3$ is the **dot product**; it measures how much two vectors point the same way. $\|\boldsymbol{a}\| = \sqrt{\boldsymbol{a}^\top \boldsymbol{a}}$ is the length. The **cross product** $\boldsymbol{a} \times \boldsymbol{b}$ is a vector at right angles to both:

Toy example — dot and cross product

$\boldsymbol{a} = [1, 0, 0]$ (east), $\boldsymbol{b} = [0, 1, 0]$ (north): $\boldsymbol{a}^\top \boldsymbol{b} = 0$ (they are at right angles), $\boldsymbol{a} \times \boldsymbol{b} = [0, 0, 1]$ (up). Rule: $\boldsymbol{a} \times \boldsymbol{b} = [a_2 b_3 - a_3 b_2,\ a_3 b_1 - a_1 b_3,\ a_1 b_2 - a_2 b_1]$.

The **identity matrix** I has ones on the diagonal and zeros elsewhere; $I\boldsymbol{a} = \boldsymbol{a}$. The **inverse** M^{-1} undoes M: $M^{-1} M = I$. $\mathrm{diag}(a, b, c)$ is a matrix with a, b, c on the diagonal and zeros elsewhere.

2.2 Rates of change and differential equations

A dot on top means "rate of change with time": $\dot{\boldsymbol{p}} = d\boldsymbol{p}/dt$ is velocity and $\ddot{\boldsymbol{p}}$ is acceleration. An equation of the form

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$$

is a **differential equation**. It does not tell you where the system is; it tells you *how fast it is changing*, given where it is ($\boldsymbol{x}$, the **state**) and what you are doing to it ($\boldsymbol{u}$, the **input**). A computer finds the motion by taking small time steps Δt.

Toy example — Euler's method for a falling ball

State $x = [\text{height};\ \text{velocity}]$, $\dot{x} = [\text{velocity};\ -9.81]$. Start at height 10 m, at rest, $\Delta t = 0.1$ s. One step: height $= 10 + 0.1 \cdot 0 = 10$, velocity $= 0 + 0.1 \cdot (-9.81) = -0.981$. Next step: height $= 10 + 0.1 \cdot (-0.981) = 9.902$ m, and so on. Each step is "new = old + $\Delta t \times$ rate". The course notes do exactly this for the Lotka–Volterra and Lorenz systems.

Euler's method is simple but not very accurate. The toy model uses **RK4** (Runge–Kutta of order 4), which looks at the rate four times inside the step (start, two in the middle, end) and averages them. MATLAB's `ode45` uses the same idea with automatic step sizes.

Discrete time. A computer works in ticks $t_k = k\,\Delta t$. A subscript k means "at tick k". So $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$ reads: next state = (matrix) × current state + (matrix) × current input. Model 2 works entirely in this form.

2.3 Taylor series and linearization

From the course notes: if we know a smooth function f and its derivatives at a point a, we can predict it nearby:

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \dots$$

Keeping only the first two terms gives the **first order (linear) approximation**: near a the curve looks like its tangent line.

Toy example — sin near 0

$f(\theta) = \sin\theta$, $a = 0$: $f(0) = 0$, $f'(0) = \cos 0 = 1$, so $\sin\theta \approx \theta$. For $\theta = 0.1$ rad: true 0.0998, linear 0.1 (0.17% off). For $\theta = 0.5$ rad: true 0.479, linear 0.5 (4.3% off). The approximation is excellent close to a and gets worse further away. This exact example decides how well the linear drone models work: θ is the tilt of the drone.

For a function of *many* variables that also gives *many* outputs, the slopes form a matrix, the **Jacobian**. Entry (i, j) is $\partial f_i/\partial x_j$: "how much output i changes when only input j is nudged". Then

$$\boldsymbol{f}(\boldsymbol{x}) \approx \boldsymbol{f}(\boldsymbol{a}) + J\,(\boldsymbol{x} - \boldsymbol{a}), \qquad J = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{a}}.$$

Toy example — a Jacobian by hand

$$\boldsymbol{f}(x,y) = \begin{bmatrix} x\,y \\ x + y^2 \end{bmatrix} \text{ at } (x,y) = (2,1): \quad J = \begin{bmatrix} \partial f_1/\partial x & \partial f_1/\partial y \\ \partial f_2/\partial x & \partial f_2/\partial y \end{bmatrix} = \begin{bmatrix} y & x \\ 1 & 2y \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix}.$$

Move to (2.1, 1): the linear prediction is $\boldsymbol{f}(2,1) + J[0.1, 0]^\top = [2, 3] + [0.1, 0.1] = [2.1, 3.1]$. The true value is $[2.1 \cdot 1,\ 2.1 + 1] = [2.1, 3.1]$. Here they agree exactly because $\boldsymbol{f}$ is linear in x.

Linear versus nonlinear. A model is *linear* if doubling the input doubles the effect, like $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$. A drone is *nonlinear* because of sin, cos and products like $\boldsymbol{\omega} \times J\boldsymbol{\omega}$. Linear models are much easier to work with, so we **linearize**: replace the nonlinear model by its Jacobian near an **operating point** such as hover.

2.4 Rotations: the rotation matrix R

The **attitude** (orientation) of the drone is stored as a 3×3 **rotation matrix** R. Its three columns are the drone's own three axes (forward, left, up), written in world coordinates:

$$R = \begin{bmatrix} \boldsymbol{b}_1 & \boldsymbol{b}_2 & \boldsymbol{b}_3 \end{bmatrix}, \qquad \boldsymbol{b}_1 = \text{drone's nose},\ \boldsymbol{b}_2 = \text{drone's left},\ \boldsymbol{b}_3 = \text{drone's top}.$$

Multiplying R by a vector written in the drone's frame gives the same vector in the world frame.

Toy example — the drone turned 90 degrees to the left (yaw)

Its nose now points north, its left points west, its top still points up: $R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$. A camera that looks along the nose, $[1, 0, 0]$ in the drone frame, looks along $R[1, 0, 0]^\top = [0, 1, 0]^\top$: north. Correct.

Every rotation matrix has two properties: the columns are unit length and at right angles, which is written $R^\top R = I$ (so $R^{-1} = R^\top$), and $\det R = +1$ (it is not a mirror). The set of all such matrices is called SO(3) ("special orthogonal group in 3D"). When you see SO(3), read "rotation matrices".

Why not roll, pitch, yaw angles? Three angles are easier to imagine, but they break down: at a pitch of 90° the roll and yaw axes line up and one direction of motion is lost (**gimbal lock**). Rotation matrices never break down. That is why our controller is called *geometric* or *SO(3)* control: it works with R directly.

2.5 The hat and vee maps

A cross product can be written as a matrix times a vector. For $\boldsymbol{a} = [a_1, a_2, a_3]$:

$$[\boldsymbol{a}]_\times = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}, \qquad [\boldsymbol{a}]_\times \boldsymbol{b} = \boldsymbol{a} \times \boldsymbol{b}.$$

$[\cdot]_\times$ is called **hat**. The matrix is *skew-symmetric*: $[\boldsymbol{a}]_\times^\top = -[\boldsymbol{a}]_\times$. The reverse operation, which reads the three numbers back out of such a matrix, is called **vee**: $[\boldsymbol{a}]_\times^\vee = \boldsymbol{a}$.

Toy example — hat and vee

$\boldsymbol{a} = [1, 2, 3]$: $[\boldsymbol{a}]_\times = \begin{bmatrix} 0 & -3 & 2 \\ 3 & 0 & -1 \\ -2 & 1 & 0 \end{bmatrix}$. Then vee takes entries (3,2), (1,3), (2,1): [1, 2, 3]. In our Python code these are `hat()` and `vee()` in `state_adapter.py`.

2.6 Exp and Log: rotation vector ↔ rotation matrix

A rotation can also be described by a **rotation vector** $\boldsymbol{\phi} = \theta\, \boldsymbol{n}$: turn by the angle θ about the unit axis $\boldsymbol{n}$. The two descriptions are linked by

$$\mathrm{Exp}(\boldsymbol{\phi}) = I + \sin\theta\, [\boldsymbol{n}]_\times + (1 - \cos\theta)[\boldsymbol{n}]_\times^2 \quad \text{(Rodrigues' formula)}, \qquad \mathrm{Log}(R) = \boldsymbol{\phi}.$$

For a *small* rotation, $\mathrm{Exp}(\boldsymbol{\phi}) \approx I + [\boldsymbol{\phi}]_\times$. This is used everywhere to describe a small attitude error with 3 numbers instead of 9.

Toy example — 90 degrees about the vertical axis

$\boldsymbol{n} = [0, 0, 1]$, $\theta = \pi/2$: $\sin\theta = 1$, $1 - \cos\theta = 1$, $[\boldsymbol{n}]_\times^2 = \mathrm{diag}(-1, -1, 0)$, so $\mathrm{Exp} = I + [\boldsymbol{n}]_\times + \mathrm{diag}(-1, -1, 0) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, which is the yaw matrix of the earlier example. And Log of that matrix gives back $[0, 0, 1.5708]$.

2.7 Uncertainty: mean, variance, covariance

Sensors are noisy, so we need words for "how unsure". The **mean** is the average. The **variance** σ^2 is the average squared distance from the mean, and the **standard deviation** σ is the typical size of the error. For a vector of errors, the **covariance matrix** $P = \mathbb{E}[\boldsymbol{e}\boldsymbol{e}^\top]$ holds the variance of each component on its diagonal and "how two components move together" off the diagonal. A **Gaussian** (bell curve) $\mathcal{N}(\mu, \sigma^2)$ puts about 68% of samples within $\pm\sigma$ and 99.7% within $\pm 3\sigma$.

Toy example — three altitude readings

Readings 9.8, 10.1, 10.1 m: mean 10.0, deviations -0.2, 0.1, 0.1, variance $(0.04 + 0.01 + 0.01)/3 = 0.02\,\mathrm{m}^2$, $\sigma = 0.14\,\mathrm{m}$.

The one rule we use again and again. If an error vector is multiplied by a matrix, $\boldsymbol{e}' = F\boldsymbol{e}$, its covariance becomes

$$P' = \mathbb{E}[F\boldsymbol{e}\boldsymbol{e}^\top F^\top] = F\,P\,F^\top.$$

Toy example — uncertainty grows when you integrate

Position and velocity errors $[\delta p, \delta v]$ with $P = \mathrm{diag}(0.01,\ 0.04)$ (10 cm and 20 cm/s). After 1 s, $\delta p' = \delta p + 1 \cdot \delta v$, $\delta v' = \delta v$, so $F = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ and $P' = FPF^\top = \begin{bmatrix} 0.05 & 0.04 \\ 0.04 & 0.04 \end{bmatrix}$. The position variance grew from 0.01 to 0.05 (σ from 10 to 22 cm), because an unknown velocity turns into an unknown position.

2.8 Finding the lowest point of a bowl

At the bottom of a smooth curve the slope is zero. For $g(u) = au^2 + bu + c$ with $a > 0$ (a bowl):

$$g'(u) = 2au + b = 0 \quad \Longrightarrow \quad u^* = -\frac{b}{2a}.$$

The many-variable version: for $g(\boldsymbol{u}) = \boldsymbol{u}^\top M\boldsymbol{u} + \boldsymbol{b}^\top\boldsymbol{u}$ with M "positive definite" (a bowl in every direction, meaning $\boldsymbol{u}^\top M\boldsymbol{u} > 0$ for every $\boldsymbol{u} \neq 0$), the gradient is $2M\boldsymbol{u} + \boldsymbol{b} = \boldsymbol{0}$, so $\boldsymbol{u}^* = -\frac{1}{2}M^{-1}\boldsymbol{b}$. Both the Kalman gain and the LQR gain are found exactly like this.

Toy example — a one-dimensional bowl

$g(u) = u^2 - 4u + 1$: $g'(u) = 2u - 4 = 0 \Rightarrow u^* = 2$, and $g(2) = -3$ is the lowest value.

2.9 Eigenvalues: will it blow up or die out?

If $M\boldsymbol{v} = \lambda\boldsymbol{v}$, the direction $\boldsymbol{v}$ is only stretched by M, by the factor λ (an **eigenvalue**). For $\boldsymbol{x}_{k+1} = M\boldsymbol{x}_k$, every such direction is multiplied by λ at every tick, so

all $|\lambda| < 1$ $\Rightarrow$ the state dies out (stable); some $|\lambda| > 1$ $\Rightarrow$ it blows up (unstable).

The largest $|\lambda|$ is called the **spectral radius** $\rho(M)$.

Toy example — two scalar systems

$x_{k+1} = 1.1\,x_k$: 1, 1.1, 1.21, 1.33, ... grows for ever. $x_{k+1} = 0.62\,x_k$: 1, 0.62, 0.38, 0.24, ... dies out. Section 6.4 shows that LQR turns the first system into the second.

For continuous-time systems $\dot{\boldsymbol{x}} = M\boldsymbol{x}$ the rule is about the real part instead: all eigenvalues with negative real part means stable.

2.10 Least squares and the pseudo-inverse

Given many noisy examples, find the one rule that fits them best. If the rule is $y \approx g\,x$ and we have samples (x_i, y_i), the **least-squares** choice minimises $\sum_i (y_i - g\,x_i)^2$. Setting the derivative to zero gives $g = \sum x_i y_i / \sum x_i^2$. For many rows at once, with all inputs as columns of Ω and all outputs as columns of Y:

$$G = Y\,\Omega^\top (\Omega\Omega^\top)^{-1} = Y\,\Omega^\dagger.$$

$\Omega^\dagger$ is the **pseudo-inverse** (MATLAB: `pinv`). It is the EDMD formula `K = Psi2*pinv(Psi1)` from the course notes, and DMDc in section 6.2.

Toy example — fitting a slope

Samples $(1, 2.1)$, $(2, 3.9)$, $(3, 6.0)$: $\sum xy = 2.1 + 7.8 + 18 = 27.9$, $\sum x^2 = 14$, so $g = 1.993$, close to the true slope 2.

3 Our Drone and Our Software

3.1 The pieces of the system

Piece	What it does for us
Gazebo + Iris model	The "real" drone: a 3D physics simulator that solves the equations of motion of a 1.5 kg quadrotor
ArduPilot SITL	The flight computer software, running on the PC. It reads the simulated IMU, runs its own estimator (EKF3) and its own low-level controllers, and drives the four motors
ROS 2	The "post office" between programs. Programs publish and subscribe to named *topics*
Monocular camera + VIO	Images → visual odometry pose; fused with the IMU it gives the state; the same poses build the 3D map
`drones_controller`	Our ROS 2 package: reads the state, runs the SO(3) controller, sends commands
`toy_model3`	Our MATLAB toy model: every equation of this document, runnable, with small examples

The ROS 2 topics that matter:

Topic	Direction	Content
`/ap/v1/pose/filtered`	ArduPilot → us	position $\boldsymbol{p}$ and orientation (quaternion → R), from ArduPilot's EKF
`/ap/v1/twist/filtered`	ArduPilot → us	velocity $\boldsymbol{v}$ and angular velocity $\boldsymbol{\omega}$
`/ap/v1/cmd_vel`	us → ArduPilot	the velocity command we want the drone to fly
`/drones_controller/*`	us → anyone	our diagnostics: $\boldsymbol{e}_p, \boldsymbol{e}_v, \boldsymbol{F}_d, \boldsymbol{\tau}, \boldsymbol{e}_R, \boldsymbol{e}_\omega, R_d$

3.2 Frames: which way is up?

- **World frame, ENU**: x = East, y = North, z = Up. ArduPilot's ROS 2 topics use this. Gravity is therefore $\boldsymbol{g} = [0,\ 0,\ -9.81]^\top$ m/s^2 (it pulls towards $-z$).
- **Body frame, FLU**: fixed to the drone, x = Forward (nose), y = Left, z = Up (through the propellers). The thrust always points along body $+z$.
- The rotation matrix R turns body-frame vectors into world-frame vectors (section 2.4).

3.3 The state of the drone

Everything we need to know about the drone at one moment is its **state**:

Symbol	Meaning	Size	Unit
$\boldsymbol{p}$	position (world)	3 numbers	m
$\boldsymbol{v}$	velocity (world)	3 numbers	m/s
R	attitude: rotation body $\rightarrow$ world	3×3 matrix	–
$\boldsymbol{\omega}$	angular velocity, measured in the body (what a gyro reads)	3 numbers	rad/s

That is 18 stored numbers but only 12 "degrees of freedom", because R has 9 entries but only 3 independent directions of rotation. The **input** is $\boldsymbol{u} = [T;\ \boldsymbol{\tau}]$: total thrust T (newtons) and the three body torques $\boldsymbol{\tau}$ (newton-metres).

A hat on a symbol means *estimated* ($\hat{\boldsymbol{p}}$ = the position the estimator believes). A subscript d means *desired* ($\boldsymbol{p}_d$ = where we want to be). A tilde means *measured by a sensor* ($\tilde{\boldsymbol{p}}$).

3.4 The parameters: `controller.yaml`

All numbers used in this document and in the toy model come from the configuration file of our controller:

drones_controller/config/controller.yaml

```yaml
so3_controller:
  ros__parameters:
    pose_topic: "/ap/v1/pose/filtered"
    twist_topic: "/ap/v1/twist/filtered"
    cmd_vel_topic: "/ap/v1/cmd_vel"
    control_rate: 25.0
    desired_x: 0.0
    desired_y: 0.0
    desired_z: 2.0
    desired_yaw: 0.0
    mass: 1.5
    gravity: 9.81
    kp: [1.5, 1.5, 2.0]
    kv: [1.2, 1.2, 1.5]
    kR: [4.0, 4.0, 2.0]
    kOmega: [0.8, 0.8, 0.5]
    inertia: [0.02, 0.02, 0.04]
    max_velocity: 1.0
    state_timeout: 0.5
    enable_command: false
```

YAML key	Symbol in this document	Value
`mass`, `gravity`	m, g_0	$1.5\,\mathrm{kg}$, $9.81\,\mathrm{m/s^2}$
`inertia`	$J = \mathrm{diag}(J_x, J_y, J_z)$	$\mathrm{diag}(0.02,\ 0.02,\ 0.04)\,\mathrm{kg\,m^2}$
`kp`, `kv`	K_p, K_v (position loop)	$\mathrm{diag}(1.5, 1.5, 2.0)$, $\mathrm{diag}(1.2, 1.2, 1.5)$
`kR`, `kOmega`	K_R, K_ω (attitude loop)	$\mathrm{diag}(4, 4, 2)$, $\mathrm{diag}(0.8, 0.8, 0.5)$
`desired_x/y/z`, `desired_yaw`	$\boldsymbol{p}_d$, ψ_d	$[0, 0, 2]\,\mathrm{m}$, $0\,\mathrm{rad}$
`control_rate`	$1/\Delta t$	$25\,\mathrm{Hz}$ ($\Delta t = 0.04\,\mathrm{s}$)
`max_velocity`	speed limit	$1.0\,\mathrm{m/s}$
`state_timeout`	how old the state may be	$0.5\,\mathrm{s}$
`enable_command`	actually send commands?	`false` by default (safety)

The rotor numbers (arm positions, thrust constant $k_f = 8.549 \times 10^{-6}\,\mathrm{N\,s^2}$, yaw constant $c_m = 0.016\,\mathrm{m}$) are those of the ArduPilot Gazebo Iris. They are only needed in the motor mixer (section 4.8).

4 Model 1: Physics, Estimation and Geometric Control

Model 1 is the loop that flies the drone (fig. 2). Blocks 1–3 describe how the drone moves and how a small error grows. Block 4 estimates the state from the sensors. Blocks 5–7 decide thrust and torque. Block 8 turns them into motor speeds. Toy model files: `M1_1_Dynamics_Linearization.mlx` (blocks 1–3), `M1_2_Error_State_EKF.mlx` (block 4) and `M1_3_SO3_Control_Closed_Loop.mlx` (blocks 5–8).

4.1 Block 1 — How the drone moves (nonlinear dynamics)

The idea in plain words

A quadrotor is one solid body. Its four propellers can only push in one direction: straight out of the top of the drone. Gravity pulls it down. To move sideways it must first *tilt*, so that part of its push points sideways. To tilt, the rotors push unevenly, which creates a twisting force (torque). So there are two connected stories: how the body *moves* (Newton) and how it *turns* (Euler).

Equations (1)–(4): rigid-body dynamics of the quadrotor

$$\dot{\boldsymbol{p}} = \boldsymbol{v} \tag{1}$$

$$\dot{\boldsymbol{v}} = \boldsymbol{g} + \frac{T}{m} R\,\boldsymbol{e}_3 \tag{2}$$

$$\dot{R} = R\,[\boldsymbol{\omega}]_\times \tag{3}$$

$$\dot{\boldsymbol{\omega}} = J^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega} \times J\boldsymbol{\omega}) \tag{4}$$

State $\boldsymbol{x} = (\boldsymbol{p}, \boldsymbol{v}, R, \boldsymbol{\omega})$, input $\boldsymbol{u} = [T;\ \boldsymbol{\tau}]$, $\boldsymbol{e}_3 = [0, 0, 1]^\top$. In short: $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$.

Where each equation comes from

(1) Position. Velocity is, by definition, how fast position changes: $\boldsymbol{v} = d\boldsymbol{p}/dt$. Nothing more.

(2) Newton's second law, $m\,\dot{\boldsymbol{v}}$ = sum of forces. Two forces act on the drone.

1. Gravity: $m\boldsymbol{g} = m[0, 0, -9.81]^\top$, always straight down.
2. Thrust: the four rotors together push with T newtons along the drone's own up-axis. In the body frame this is $T\boldsymbol{e}_3 = [0, 0, T]^\top$. To write it in the world frame we multiply by R (section 2.4): $RT\boldsymbol{e}_3 = T\,\boldsymbol{b}_3$, where $\boldsymbol{b}_3$ is the third column of R, the direction the top of the drone points.

So $m\dot{\boldsymbol{v}} = m\boldsymbol{g} + TR\boldsymbol{e}_3$. Dividing by m gives (2). Air drag and wind are left out of the model. In the toy simulation they are added as unknown "gusts", which is exactly what the controller has to cope with.

(3) How the attitude changes. Take a point fixed on the drone, at $\boldsymbol{r}$ in body coordinates. In the world it is at $R\boldsymbol{r}$, so its velocity is $\dot{R}\boldsymbol{r}$. A body spinning with angular velocity $\boldsymbol{\omega}$ (measured in the body) moves that point with velocity $\boldsymbol{\omega} \times \boldsymbol{r}$ in body coordinates, which is $R(\boldsymbol{\omega} \times \boldsymbol{r}) = R[\boldsymbol{\omega}]_\times \boldsymbol{r}$ in the world. Since this holds for every $\boldsymbol{r}$: $\dot{R} = R[\boldsymbol{\omega}]_\times$. This is how the gyroscope reading $\boldsymbol{\omega}$ turns the attitude.

(4) Euler's law for rotation. The "spin momentum" (angular momentum) is $J\boldsymbol{\omega}$, where J is the inertia matrix (how hard the body is to spin about each axis). Torque changes angular momentum. Because the body frame itself is turning, an extra term appears:

$$J\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times J\boldsymbol{\omega} = \boldsymbol{\tau} \quad \Longrightarrow \quad \dot{\boldsymbol{\omega}} = J^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega} \times J\boldsymbol{\omega}).$$

The term $\boldsymbol{\omega} \times J\boldsymbol{\omega}$ is the **gyroscopic coupling**: spinning about two axes at once starts a spin about the third axis, even with no torque. (Derivation: the world-frame momentum is $\boldsymbol{h} = RJ\boldsymbol{\omega}$; Newton for rotation says $\dot{\boldsymbol{h}} = R\boldsymbol{\tau}$; differentiating with (3) gives $R([\boldsymbol{\omega}]_\times J\boldsymbol{\omega} + J\dot{\boldsymbol{\omega}}) = R\boldsymbol{\tau}$, which is the equation above.)

Toy example — hover, a 10 degree tilt, and gyroscopic coupling ($m = 1.5\,\text{kg}$, $J = \text{diag}(0.02, 0.02, 0.04)$)

1. Hover. Level ($R = I$), not spinning, thrust equal to the weight $T = mg_0 = 1.5 \times 9.81 = 14.715\,\text{N}$:

$$\dot{\boldsymbol{v}} = \begin{bmatrix} 0 \\ 0 \\ -9.81 \end{bmatrix} + \frac{14.715}{1.5} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

No acceleration: the drone hangs still. This balance point is called the **hover equilibrium**.

2. Rolled by 10°, same thrust. The top of the drone now points along $R\boldsymbol{e}_3 = [0,\ -\sin 10^\circ,\ \cos 10^\circ] = [0,\ -0.1736,\ 0.9848]$:

$$\dot{\boldsymbol{v}} = \begin{bmatrix} 0 \\ 0 \\ -9.81 \end{bmatrix} + 9.81 \begin{bmatrix} 0 \\ -0.1736 \\ 0.9848 \end{bmatrix} = \begin{bmatrix} 0 \\ -1.7035 \\ -0.1490 \end{bmatrix} \text{ m/s}^2.$$

It accelerates sideways (this is how a drone moves sideways!) and also sinks a little, because only 98.5% of the thrust still holds it up.

3. Gyroscopic coupling. Spinning at $\boldsymbol{\omega} = [1, 0, 1]\,\text{rad/s}$ with no torque: $J\boldsymbol{\omega} = [0.02,\ 0,\ 0.04]$, $\boldsymbol{\omega} \times J\boldsymbol{\omega} = [0 \cdot 0.04 - 1 \cdot 0,\ 1 \cdot 0.02 - 1 \cdot 0.04,\ 0] = [0,\ -0.02,\ 0]$, so $\dot{\boldsymbol{\omega}} = -J^{-1}[0, -0.02, 0] = [0,\ 1,\ 0]\,\text{rad/s}^2$. A spin about y appears on its own.

Toy model: `toy_model3/M1_1_Dynamics_Linearization.mlx`, section "Block 1"

```
x_hover = DroneLib.pack([0;0;2], [0;0;0], eye(3), [0;0;0]);
u_hover = [m*P.g0; 0; 0; 0]          % [T; tau_x; tau_y; tau_z]
xdot    = DroneLib.f(x_hover, u_hover, P);
```

```
v_dot = xdot(4:6)'                    % zero : the drone hangs still
```

The function `DroneLib.f` is equations (1)–(4) written in MATLAB, line by line:

toy_model3/DroneLib.m, function f

```
v_dot = P.g + u(1)/P.m*R*P.e3 + d(1:3)/P.m;          % (2) + wind force d
R_dot = R*DroneLib.hat(w);                            % (3)
w_dot = P.J \ (u(2:4) - cross(w, P.J*w) + d(4:6));    % (4) + disturbance torque
xdot  = [v; v_dot; R_dot(:); w_dot];                  % (1): p_dot = v
```

A drone cannot fly by itself. The toy model gives the hovering drone a tiny roll torque of 0.01 N m for just 0.1 s and then lets it go with hover thrust (solved with `ode45`, as in the course notes). Nobody corrects it, and after 4 s it is **4.93 m** away from where it started (fig. 4). A quadrotor is *unstable*: it needs to know its state (block 4) and it needs feedback (blocks 5–8).

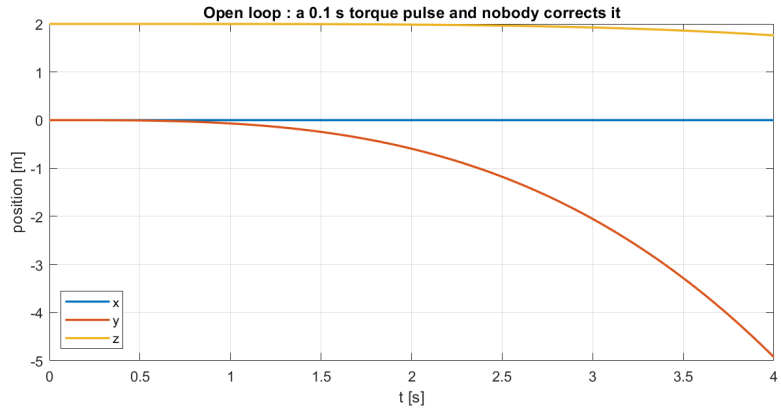


Figure 4: Open loop: a 0.1 s torque pulse and no correction. The drone slides further and further away.

How this controls the real drone

Equations (1)–(4) are "the real drone". In our setup Gazebo solves them for the Iris quadrotor; we never solve them in the controller. Our node only *reads* their result. The position and the attitude arrive on `/ap/v1/pose/filtered`; the attitude comes as a quaternion and is turned into the rotation matrix R:

drones_controller/controller_node.py — pose_callback()

```
371     def pose_callback(self, msg):
372         self.position = np.array([
373             msg.pose.position.x,
374             msg.pose.position.y,
375             msg.pose.position.z
376         ])
377         try:
378             self.rotation = quaternion_to_rotation_matrix(
379                 msg.pose.orientation.x,
380                 msg.pose.orientation.y,
381                 msg.pose.orientation.z,
382                 msg.pose.orientation.w
383             )
```

drones_controller/state_adapter.py — quaternion_to_rotation_matrix(): the matrix R

```
24     R = np.array([
25         [
```

```
            1.0 - 2.0 * (y*y + z*z),
            2.0 * (x*y - z*w),
            2.0 * (x*z + y*w)
        ],
        [
            2.0 * (x*y + z*w),
            1.0 - 2.0 * (x*x + z*z),
            2.0 * (y*z - x*w)
        ],
        [
            2.0 * (x*z - y*w),
            2.0 * (y*z + x*w),
            1.0 - 2.0 * (x*x + y*y)
        ]
    ])
```

The node also checks that this really is a rotation matrix ($R^\top R = I$, $\det R = 1$), the two properties of section 2.4:

drones_controller/state_adapter.py — validate_rotation_matrix()

```
def validate_rotation_matrix(R):
    orthogonality_error = np.linalg.norm(
        R.T @ R - np.eye(3)
    )
    determinant_error = abs(np.linalg.det(R) - 1.0)
    return (
        orthogonality_error < 1e-3
        and determinant_error < 1e-3
    )
```

4.2 Block 2 — Linearization: zooming in on the dynamics

The idea in plain words

Equations (1)–(4) are curved (nonlinear): they contain R, products and sines. But if we zoom in close to one flight condition, any smooth curve looks like a straight line (section 2.3). Near that condition the drone behaves like a simple *linear* system, and linear systems are easy to predict and to control. The "slopes" of this straight-line model are two matrices, A and B.

Equations (5)–(6): first order Taylor expansion and the Jacobians

$$\boldsymbol{f}(\boldsymbol{x},\boldsymbol{u}) \approx \boldsymbol{f}(\hat{\boldsymbol{x}},\hat{\boldsymbol{u}}) + A\,\delta\boldsymbol{x} + B\,\delta\boldsymbol{u} \tag{5}$$

$$A = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\hat{\boldsymbol{x}},\hat{\boldsymbol{u}}} \in \mathbb{R}^{12\times 12}, \qquad B = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\right|_{\hat{\boldsymbol{x}},\hat{\boldsymbol{u}}} \in \mathbb{R}^{12\times 4} \tag{6}$$

with the 12-number error $\delta\boldsymbol{x} = [\delta\boldsymbol{p};\ \delta\boldsymbol{v};\ \delta\boldsymbol{\theta};\ \delta\boldsymbol{\omega}]$, where $R = \hat{R}\,\mathrm{Exp}(\delta\boldsymbol{\theta})$, and $\delta\boldsymbol{u} = \boldsymbol{u} - \hat{\boldsymbol{u}}$.

What is δx?

$\delta\boldsymbol{x}$ is "the difference between the true state and a reference state" (later: the estimate). For position, velocity and angular velocity this is a simple subtraction: $\delta\boldsymbol{p} = \boldsymbol{p} - \hat{\boldsymbol{p}}$ and so on. For the attitude

we cannot subtract matrices (the result would not be a rotation). Instead we ask: "which small rotation $\delta\boldsymbol{\theta}$ takes $\hat{R}$ to R?", i.e. $R = \hat{R}\,\mathrm{Exp}(\delta\boldsymbol{\theta})$, so $\delta\boldsymbol{\theta} = \mathrm{Log}(\hat{R}^\top R)$. That is 3 numbers. So $\delta\boldsymbol{x}$ has $3+3+3+3 = 12$ numbers.

Deriving A and B, one row at a time

Put $\boldsymbol{p} = \hat{\boldsymbol{p}} + \delta\boldsymbol{p}$, $\boldsymbol{v} = \hat{\boldsymbol{v}} + \delta\boldsymbol{v}$, $R \approx \hat{R}(I + [\delta\boldsymbol{\theta}]_\times)$, $\boldsymbol{\omega} = \hat{\boldsymbol{\omega}} + \delta\boldsymbol{\omega}$, $T = \hat{T} + \delta T$, $\boldsymbol{\tau} = \hat{\boldsymbol{\tau}} + \delta\boldsymbol{\tau}$ into (1)–(4). Keep every term that contains *one* small quantity and throw away products of two small quantities (they are tiny squared).

Position row. $\dot{\boldsymbol{p}} = \boldsymbol{v}$ gives $\delta\dot{\boldsymbol{p}} = \delta\boldsymbol{v}$ directly.

Velocity row. $\dot{\boldsymbol{v}} \approx \boldsymbol{g} + \frac{\hat{T}+\delta T}{m}\hat{R}(I + [\delta\boldsymbol{\theta}]_\times)\boldsymbol{e}_3$. Multiply out and subtract the reference part $\boldsymbol{g} + \frac{\hat{T}}{m}\hat{R}\boldsymbol{e}_3$:

$$\delta\dot{\boldsymbol{v}} = \frac{\hat{T}}{m}\hat{R}[\delta\boldsymbol{\theta}]_\times\boldsymbol{e}_3 + \frac{\delta T}{m}\hat{R}\boldsymbol{e}_3 + \underbrace{\tfrac{\delta T}{m}\hat{R}[\delta\boldsymbol{\theta}]_\times\boldsymbol{e}_3}_{\text{two small terms: drop}}.$$

Using $[\delta\boldsymbol{\theta}]_\times\boldsymbol{e}_3 = \delta\boldsymbol{\theta}\times\boldsymbol{e}_3 = -\boldsymbol{e}_3\times\delta\boldsymbol{\theta} = -[\boldsymbol{e}_3]_\times\delta\boldsymbol{\theta}$:

$$\delta\dot{\boldsymbol{v}} = -\frac{\hat{T}}{m}\hat{R}[\boldsymbol{e}_3]_\times\,\delta\boldsymbol{\theta} + \frac{1}{m}\hat{R}\boldsymbol{e}_3\,\delta T.$$

Attitude row. Both R and $\hat{R}$ follow (3). Differentiating $R = \hat{R}\mathrm{Exp}(\delta\boldsymbol{\theta})$ and keeping first order terms gives

$$\delta\dot{\boldsymbol{\theta}} = -[\hat{\boldsymbol{\omega}}]_\times\,\delta\boldsymbol{\theta} + \delta\boldsymbol{\omega}.$$

In words: an angular-velocity error makes the attitude error grow; the first term just accounts for the reference frame itself spinning.

Angular-velocity row. The only nonlinear part of (4) is $\boldsymbol{\omega}\times J\boldsymbol{\omega}$. Its derivative with respect to $\boldsymbol{\omega}$ is $[\boldsymbol{\omega}]_\times J - [J\boldsymbol{\omega}]_\times$ (product rule for the cross product), so

$$\delta\dot{\boldsymbol{\omega}} = J^{-1}\big([J\hat{\boldsymbol{\omega}}]_\times - [\hat{\boldsymbol{\omega}}]_\times J\big)\delta\boldsymbol{\omega} + J^{-1}\delta\boldsymbol{\tau}.$$

Stacking the four rows (each block is 3×3; the columns follow $\delta\boldsymbol{p}, \delta\boldsymbol{v}, \delta\boldsymbol{\theta}, \delta\boldsymbol{\omega}$ for A and $\delta T, \delta\boldsymbol{\tau}$ for B):

$$A = \begin{bmatrix} 0 & I & 0 & 0 \\ 0 & 0 & -\frac{\hat{T}}{m}\hat{R}[\boldsymbol{e}_3]_\times & 0 \\ 0 & 0 & -[\hat{\boldsymbol{\omega}}]_\times & I \\ 0 & 0 & 0 & J^{-1}\big([J\hat{\boldsymbol{\omega}}]_\times - [\hat{\boldsymbol{\omega}}]_\times J\big) \end{bmatrix}, \qquad B = \begin{bmatrix} 0 & 0 \\ \frac{1}{m}\hat{R}\boldsymbol{e}_3 & 0 \\ 0 & 0 \\ 0 & J^{-1} \end{bmatrix}. \tag{6a}$$

Toy example — the Jacobians at hover, and how good "first order" is

At hover ($\hat{R} = I$, $\hat{\boldsymbol{\omega}} = 0$, $\hat{T} = mg_0$) the important blocks are

$$-g_0[\boldsymbol{e}_3]_\times = \begin{bmatrix} 0 & 9.81 & 0 \\ -9.81 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad \tfrac{1}{m}\boldsymbol{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 0.667 \end{bmatrix}, \qquad J^{-1} = \mathrm{diag}(50,\,50,\,25).$$

Read them as sentences:

- a pitch error of $\delta\theta_y$ radians accelerates the drone along $+x$ at $9.81\,\delta\theta_y\,\mathrm{m/s^2}$ (row 1 of the first matrix); a roll error accelerates it along $-y$ (row 2);
- 1 N of extra thrust gives $0.667\,\mathrm{m/s^2}$ of extra *vertical* acceleration and nothing else;

- 1 N m of roll torque gives 50 rad/s^2 of roll acceleration.

Check by numbers. The toy model also computes the derivatives numerically, $(\boldsymbol{f}(\boldsymbol{x}+h) - \boldsymbol{f}(\boldsymbol{x}-h))/2h$, at a tilted, spinning, climbing state. The largest difference from the formulas is 4.9×10^{-10} for A and 6.8×10^{-10} for B: the derivation is right.

How far can we trust the straight line? Pitch the hovering drone by θ. The linear model says $\dot{v}_x = g_0\theta$, the truth is $g_0 \sin\theta$ (the sin example of section 2.3):

θ [deg]	linear $g_0\theta$ [m/s^2]	true $g_0 \sin\theta$ [m/s^2]	error
1	0.1712	0.1712	0.005%
2.87	0.4914	0.4912	0.04%
10	1.7122	1.7035	0.51%
20	3.4243	3.3552	2.06%
28.65	4.9054	4.7035	4.29%

In normal flight the tilt stays below 10°, where the linear model is better than 0.5%.

Toy model: `toy_model3/M1_1_Dynamics_Linearization.mlx`, section "Block 2"

```
[A, B] = DroneLib.jacobians(x_hover, u_hover, P);
A_v_theta = A(4:6, 7:9)      % a pitch error pushes along +x, a roll error along -y
B_v_T     = B(4:6, 1)'       % extra thrust only changes vertical acceleration (1/m)
B_w_tau   = B(10:12, 2:4)    % torque -> angular acceleration (J^-1)
```

How this controls the real drone

Our Python node does not compute A and B; they are used *inside the estimator* (block 4), which in our SITL setup is ArduPilot's EKF3. They are also the reason Model 2 can use a linear model at all: the table above shows that near hover the drone really is almost linear. And they are what Model 2 learns from data (section 6.2): DMDc recovers these same numbers without knowing m or J.

4.3 Block 3 — How a small error grows (local error dynamics)

The idea in plain words

Suppose our belief about the drone is slightly wrong. How does that mistake change over time if we only trust the physics? Block 3 answers this with the linear model of block 2. This is exactly what the estimator needs to know to decide how much it can still trust its own prediction.

Equations (7)–(9): linear dynamics of the error

$$\delta\boldsymbol{x} = [\boldsymbol{p} - \hat{\boldsymbol{p}};\ \boldsymbol{v} - \hat{\boldsymbol{v}};\ \mathrm{Log}(\hat{R}^\top R);\ \boldsymbol{\omega} - \hat{\boldsymbol{\omega}}] \tag{7}$$

$$\delta\dot{\boldsymbol{x}} = A\,\delta\boldsymbol{x} + B\,\delta\boldsymbol{u} \tag{8}$$

$$\delta\boldsymbol{x}_{k+1} = F\,\delta\boldsymbol{x}_k + G\,\delta\boldsymbol{u}_k, \qquad F = e^{A\Delta t} \approx I + A\Delta t, \quad G \approx B\Delta t \tag{9}$$

Derivation

The truth follows $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$ and the estimate follows the same model, $\dot{\hat{\boldsymbol{x}}} = \boldsymbol{f}(\hat{\boldsymbol{x}}, \hat{\boldsymbol{u}})$. Subtract the two and use (5):

$$\delta\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}) - \boldsymbol{f}(\hat{\boldsymbol{x}}, \hat{\boldsymbol{u}}) \approx \big[\boldsymbol{f}(\hat{\boldsymbol{x}}, \hat{\boldsymbol{u}}) + A\delta\boldsymbol{x} + B\delta\boldsymbol{u}\big] - \boldsymbol{f}(\hat{\boldsymbol{x}}, \hat{\boldsymbol{u}}) = A\delta\boldsymbol{x} + B\delta\boldsymbol{u}.$$

The big nonlinear parts cancel; only the small linear error is left. That is (8).

For a computer we need one time step Δt. The exact solution of $\delta\dot{\boldsymbol{x}} = A\delta\boldsymbol{x}$ over one step is $\delta\boldsymbol{x}_{k+1} = e^{A\Delta t}\delta\boldsymbol{x}_k$, where the matrix exponential is the series $e^{A\Delta t} = I + A\Delta t + \frac{1}{2}(A\Delta t)^2 + \dots$ (Taylor again). Keeping the first two terms gives $F \approx I + A\Delta t$, which is the same as one Euler step.

Why a drone drifts away. At hover every eigenvalue of A is 0. Look at the chain inside A: angular-velocity error → attitude error → velocity error → position error. Each arrow is an integration. A constant roll-rate error $\delta\omega_x$ therefore becomes $\delta\theta_x = \delta\omega_x t$, then $\delta v_y = -g_0\delta\omega_x t^2/2$, then

$$\delta y(t) = -\frac{g_0\,\delta\omega_x\,t^3}{6}.$$

Toy example — a tiny gyro error becomes a big position error

$\delta\omega_x = 0.01\,\text{rad/s}$ (about half a degree per second, a typical gyro bias). After 2 s: $\delta y = -9.81 \times 0.01 \times 8/6 = -0.131\,\text{m}$. After 5 s: $-2.04\,\text{m}$. After 10 s: $-16.4\,\text{m}$. This t^3 growth is why an IMU on its own is useless for position after a few seconds, and why the camera is needed.

Linear prediction versus truth. The toy model starts the drone with an initial error (10 cm, 0.2 rad/s of yaw rate, and a roll error) and compares the linear prediction $e^{At}\delta\boldsymbol{x}_0$ with the true nonlinear motion over 2 s. With a 2° roll error they differ by only 0.02%; with a 30° roll error the linear model is off by 4.7% (fig. 5). The linear model is *local*: perfect for small errors, worse for large ones.

One-step check. With $\Delta t = 0.01\,\text{s}$, the biggest entry of $F - e^{A\Delta t}$ is 4.9×10^{-4}, the size of the dropped $\frac{1}{2}(A\Delta t)^2$ term. So $F = I + A\Delta t$ is good enough.

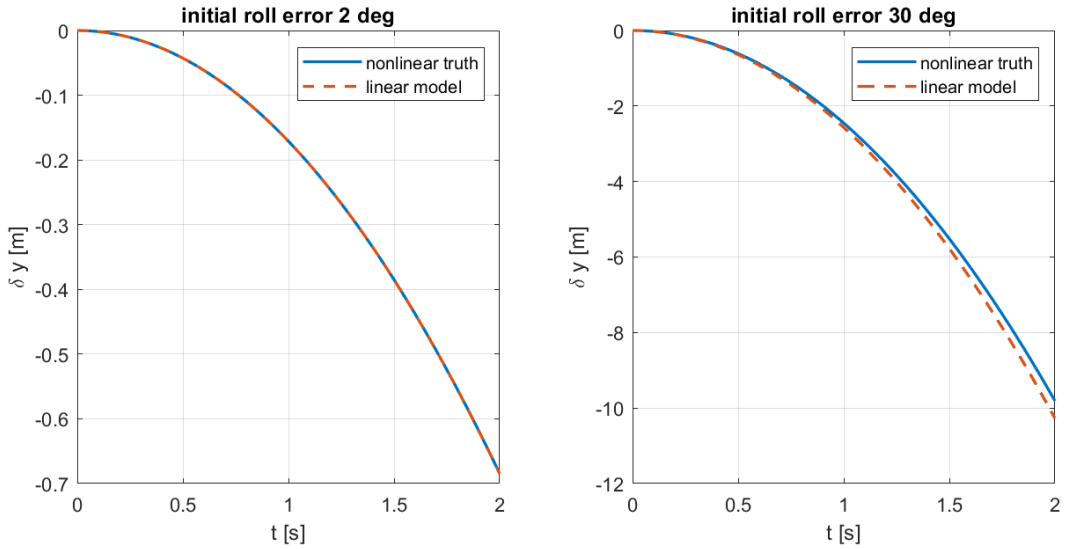


Figure 5: Sideways error δy after an initial roll error: nonlinear truth (solid) and linear model (dashed).

Toy model: `toy_model3/M1_1_Dynamics_Linearization.mlx`, section "Block 3"

```
dt = 0.01;                          % EKF / IMU sample time [s]
F  = eye(12) + A*dt;
```

```
max_F_error = max(abs(F - expm(A*dt)), [], 'all')   % size of the dropped (A dt)^2/2
eig_A = eig(A)'                                      % all zero at hover
```

How this controls the real drone

The estimator never tries to describe the whole flight with a linear model. It only describes the *small difference* between what it believes and what is true, and that difference stays small because the camera and the IMU keep correcting it. That is why the filter of block 4 is called an *error-state* filter, and why F is all it needs.

4.4 Block 4 — Where am I? The error-state Kalman filter (our VIO)

The idea in plain words

We have two imperfect sources of information. The *physics* (blocks 1–3) predicts where the drone should be, but small mistakes grow quickly. The *sensors* tell us where it is, but every reading is noisy, and the camera only gives a new reading 25 times a second. The Kalman filter combines them in the best possible way. It repeats two steps for ever:

1. **Predict.** "I know how I pushed the drone, so physics tells me where it should be now." The guess moves forward, and because physics is never perfect, the uncertainty *grows*.
2. **Correct.** "A sensor reading arrived. How surprised am I?" The guess moves towards the reading. How far depends on whom we trust more. The uncertainty *shrinks*.

The filter remembers two things: the **estimate** $\hat{\boldsymbol{x}}_k$ (its best guess) and the **covariance** P_k (a 12×12 matrix that says how unsure it is in each of the 12 error directions).

Equations (10)–(14): error-state extended Kalman filter

Prediction (every IMU sample, 100 Hz)

$$\hat{\boldsymbol{x}}_{k+1} = \boldsymbol{f}_d(\hat{\boldsymbol{x}}_k, \boldsymbol{u}_k) \tag{10}$$

$$P_{k+1} = F P_k F^\top + Q \tag{11}$$

Correction (whenever a measurement $\boldsymbol{z}$ arrives)

$$K = P H^\top (H P H^\top + R_n)^{-1} \tag{12}$$

$$\delta\hat{\boldsymbol{x}} = K(\boldsymbol{z} - h(\hat{\boldsymbol{x}})), \quad \hat{\boldsymbol{p}} \leftarrow \hat{\boldsymbol{p}} + \delta\hat{\boldsymbol{p}},\ \hat{\boldsymbol{v}} \leftarrow \hat{\boldsymbol{v}} + \delta\hat{\boldsymbol{v}},\ \hat{R} \leftarrow \hat{R}\,\mathrm{Exp}(\delta\hat{\boldsymbol{\theta}}),\ \hat{\boldsymbol{\omega}} \leftarrow \hat{\boldsymbol{\omega}} + \delta\hat{\boldsymbol{\omega}} \tag{13}$$

$$P \leftarrow (I - KH)\,P \tag{14}$$

$\boldsymbol{f}_d$ = one simulation step of (1)–(4); Q = how much the physics may be wrong per step (wind, model errors); $\boldsymbol{z}$ = measurement; $h(\hat{\boldsymbol{x}})$ = what we expect to measure if our estimate were right; H = Jacobian of h; R_n = covariance of the sensor noise; K = Kalman gain.

First understand it with one number

Before matrices, take a single quantity, the altitude z, with a guess $\hat{z}$ and variance P.

Predict. If we command a climb of c m/s for Δt seconds, the guess becomes $\hat{z} + c\Delta t$. The old uncertainty is still there, and the physics adds a bit more (Q): $P^- = P + Q$.

Correct. A sensor reads z_m with noise variance R_n. The surprise (the **innovation**) is $r = z_m - \hat{z}^-$.

We move a fraction K of the surprise: $\hat{z}^+ = \hat{z}^- + K\,r$. Which fraction is best? The new error is $(1-K)\times$(old error) $-K\times$(sensor noise), whose variance is

$$P^+ = (1-K)^2 P^- + K^2 R_n.$$

This is a bowl in K (section 2.8). Set the slope to zero: $-2(1-K)P^- + 2KR_n = 0 \Rightarrow K = \dfrac{P^-}{P^- + R_n}$. Putting it back gives $P^+ = (1-K)P^-$.

So K is simply "my uncertainty divided by (my uncertainty + the sensor's uncertainty)". If I am very unsure (P^- large), $K \to 1$ and I believe the sensor. If the sensor is very noisy (R_n large), $K \to 0$ and I ignore it.

Toy example — one predict and one correct, altitude only

Prior: $\hat{z}_k = 10\,\text{m}$ with $P_k = 3.5\,\text{m}^2$. We command a climb of $0.5\,\text{m/s}$ for $1\,\text{s}$, $F = 1$, $Q = 0.5$:

$$\hat{z}^- = 10 + 0.5 = 10.5\ \text{m}, \qquad P^- = 1 \cdot 3.5 \cdot 1 + 0.5 = 4.0\ \text{m}^2.$$

A sensor reads $z = 12.5\,\text{m}$ with $R_n = 1\,\text{m}^2$ and $H = 1$:

$$K = \frac{4.0}{4.0+1} = 0.8, \qquad \hat{z}^+ = 10.5 + 0.8\,(12.5 - 10.5) = 12.1\,\text{m}, \qquad P^+ = (1-0.8)\cdot 4.0 = 0.8\,\text{m}^2.$$

The sensor is four times more trustworthy than the prediction (4.0 versus 1), so the estimate moves 80% of the way towards it, and the variance drops from 4.0 to 0.8. Both sources together are better than either alone.

Toy model: `toy_model3/M1_2_Error_State_EKF.mlx`, section "Toy example - altitude only"

```
z_hat = 10;  Pk = 3.5;  F = 1;  Q = 0.5;  climb = 0.5;  dt = 1;
z_hat = z_hat + climb*dt          % predict the guess
Pk    = F*Pk*F' + Q               % uncertainty grows
z = 12.5;  H = 1;  Rn = 1;
K     = Pk*H'/(H*Pk*H' + Rn)      % how much to trust the sensor
z_hat = z_hat + K*(z - H*z_hat)   % corrected guess
Pk    = (1 - K*H)*Pk              % uncertainty shrinks
```

The same thing with matrices

Now the error is the 12-vector $\delta\boldsymbol{x}$ and the uncertainty is the matrix P.

(10) Predict the guess. The best guess of the next state is to run the full nonlinear physics one step forward from the current guess with the input we applied. The error we do not know averages to zero, so it does not move the guess.

(11) Predict the uncertainty. From block 3, the error moves as $\delta\boldsymbol{x}_{k+1} = F\delta\boldsymbol{x}_k + \boldsymbol{w}_k$, where $\boldsymbol{w}_k$ is the new random error added by the wind and by model mistakes, with covariance Q. Using the rule $P' = FPF^\top$ of section 2 and the fact that $\boldsymbol{w}_k$ is independent of the old error:

$$P_{k+1} = \mathbb{E}[\delta\boldsymbol{x}_{k+1}\delta\boldsymbol{x}_{k+1}^\top] = F\,P_k\,F^\top + Q.$$

Uncertainty is stretched by the physics ($FPF^\top$) and inflated by the noise ($+Q$).

(12) The Kalman gain. A sensor does not see the whole state. A camera sees position and attitude; a gyro sees angular velocity. H picks out what the sensor sees: the innovation is $\boldsymbol{r} = \boldsymbol{z} - h(\hat{\boldsymbol{x}}) \approx H\delta\boldsymbol{x} + \boldsymbol{v}$ ("what I see minus what I expected = my error seen through the sensor + sensor noise"). Correcting by $\delta\hat{\boldsymbol{x}} = K\boldsymbol{r}$ leaves the error $(I-KH)\delta\boldsymbol{x} - K\boldsymbol{v}$ with covariance

$$P^+ = (I-KH)P(I-KH)^\top + KR_nK^\top.$$

Exactly as in the one-number case, we choose K to make the total uncertainty (the sum of the diagonal, the trace) as small as possible. Setting the derivative with respect to K to zero:

$$-2PH^\top + 2K(HPH^\top + R_n) = 0 \quad \Longrightarrow \quad K = PH^\top(HPH^\top + R_n)^{-1}.$$

Compare with $K = P^-/(P^- + R_n)$: it is the same formula, with matrices.

(13) Correct the guess. $\delta\hat{\boldsymbol{x}} = K\boldsymbol{r}$ is our best estimate of the error, so we add it to the guess. Position, velocity and angular velocity are added normally. The attitude is *rotated*: $\hat{R} \leftarrow \hat{R}\mathrm{Exp}(\delta\hat{\boldsymbol{\theta}})$, so $\hat{R}$ stays a true rotation matrix. This is the "error-state" trick: the filter works with a small 3-number attitude error and never adds matrices that should be rotations.

(14) Shrink the uncertainty. Putting the best K into the expression for P^+ simplifies it to $P^+ = (I - KH)P$.

Our sensors

Sensor (rate)	innovation $\boldsymbol{r}$	H	noise σ
Gyroscope (100 Hz)	$\tilde{\boldsymbol{\omega}} - \hat{\boldsymbol{\omega}}$	$[0 \;\; 0 \;\; 0 \;\; I]$	0.01 rad/s
Camera / VIO (25 Hz)	$\begin{bmatrix} \tilde{\boldsymbol{p}} - \hat{\boldsymbol{p}} \\ \mathrm{Log}(\hat{R}^\top \tilde{R}) \end{bmatrix}$	$\begin{bmatrix} I & 0 & 0 & 0 \\ 0 & 0 & I & 0 \end{bmatrix}$	5 cm, 0.02 rad

Each block of H is 3×3, in the order $[\delta\boldsymbol{p}, \delta\boldsymbol{v}, \delta\boldsymbol{\theta}, \delta\boldsymbol{\omega}]$. The **velocity is never measured**. The filter works it out from how the measured position changes, through the physics in F.

Toy example — the full 12-state filter on the drone (`M1_2`)

The drone flies a climbing helix in gusty wind for 20 s. The filter starts with a wrong guess (30 cm, 0.2 m/s and about 7° off) and sees only the commands and the noisy sensors. After the first 2 s:

quantity	estimation error (RMS)	for comparison
position	2.55 cm	the camera alone: 8.55 cm
velocity (never measured!)	3.85 cm/s	–
attitude	0.20°	camera noise: 1.15°

The fused estimate is more than three times better than the camera alone. In 99% of the samples all 12 errors lie inside the $\pm 3\sigma$ band that the filter itself predicts from P_k: the filter knows how unsure it is.

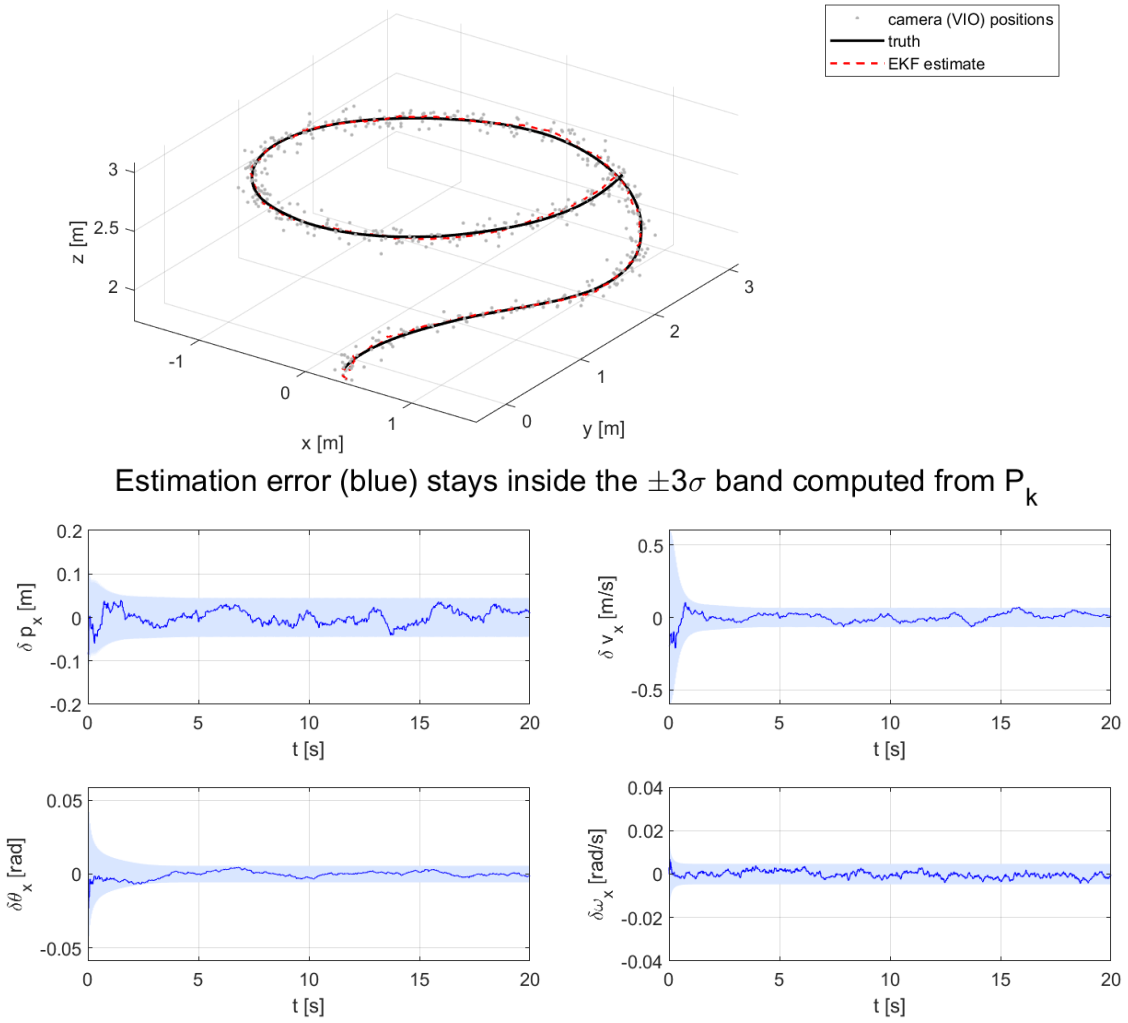


Figure 6: Top: noisy camera positions (grey dots), truth (black) and EKF estimate (red, dashed). Bottom: estimation errors (blue) stay inside the $\pm 3\sigma$ band computed from P_k.

Toy model: `toy_model3/M1_2_Error_State_EKF.mlx`, section "Run the filter"

```
for k = 1:N
    [~, ~, ~, wh] = DroneLib.unpack(xh);                          % gyro correction
    [xh, Pk] = DroneLib.ekf_update(xh, Pk, Zg(:,k) - wh, Hg, Rg);
    if mod(k-1, 4) == 0                                    % camera (25 Hz)
        [ph, ~, Rh] = DroneLib.unpack(xh);
        r = [Zp(:,k) - ph; DroneLib.logSO3(Rh'*DroneLib.expSO3(Zth(:,k)))];
        [xh, Pk] = DroneLib.ekf_update(xh, Pk, r, Hc, Rc);
    end
    ...
    [xh, Pk] = DroneLib.ekf_predict(xh, Pk, U(:,k), dt, P, Q);  % prediction
end
```

toy_model3/DroneLib.m — ekf_predict and ekf_update, equations (9)–(14)

```
A  = DroneLib.jacobians(xh, u, P);
F  = eye(12) + A*dt;                                  % (9)
xh = DroneLib.rk4(xh, u, dt, P);                      % (10)
Pk = F*Pk*F' + Q;                                     % (11)

K  = Pk*H' / (H*Pk*H' + Rn);                          % (12)
xh = DroneLib.boxplus(xh, K*r);                       % (13)  R <- R*Exp(dtheta)
```

```
Pk = (eye(12) - K*H)*Pk;                                    % (14)
```

How this controls the real drone — this is the "VIO" of our title

A monocular camera alone cannot tell scale (a small room close up looks like a big room far away) and an IMU alone drifts within seconds (t^3, block 3). The Kalman filter is the referee that fuses them. Between camera frames it predicts with the physics and the gyro; when a frame arrives, the visual-odometry pose corrects the prediction, and K decides how much to trust each source.

In our SITL setup this filter runs *inside ArduPilot* (its EKF3 has exactly this predict / correct structure, with more states such as sensor biases). Its output arrives on `/ap/v1/pose/filtered` and `/ap/v1/twist/filtered`. Our node stores it, and before every control step checks that the estimate is complete, recent and finite:

drones_controller/controller_node.py — state_is_valid() (first part)

```
447     def state_is_valid(self):
448         if (
449             self.position is None
450             or self.velocity is None
451             or self.rotation is None
452             or self.omega is None
453         ):
454             return False
455         if (
456             self.last_pose_time is None
457             or self.last_twist_time is None
458         ):
459             return False
460         now = self.get_clock().now()
461         pose_age = (
462             now - self.last_pose_time
463         ).nanoseconds * 1e-9
464         twist_age = (
465             now - self.last_twist_time
466         ).nanoseconds * 1e-9
467         if pose_age > self.state_timeout:
468             return False
469         if twist_age > self.state_timeout:
470             return False
```

If the estimate is older than `state_timeout` $= 0.5\,\text{s}$, the controller does nothing at all. This is a safety rule: never fly on stale information.

4.5 Block 5 — Position control (outer loop)

The idea in plain words

Compare where we are with where we should be, and ask for an acceleration that pulls us there: the further away, the harder we pull (a spring), and the faster we are already moving towards the target, the less we pull (a damper).

Equations (15)–(16): tracking errors and desired acceleration

$$\boldsymbol{e}_p = \boldsymbol{p}_d - \hat{\boldsymbol{p}}, \qquad \boldsymbol{e}_v = \boldsymbol{v}_d - \hat{\boldsymbol{v}} \tag{15}$$

$$\boldsymbol{a}_d = \ddot{\boldsymbol{p}}_d + K_p\,\boldsymbol{e}_p + K_v\,\boldsymbol{e}_v \tag{16}$$

$\boldsymbol{p}_d, \boldsymbol{v}_d, \ddot{\boldsymbol{p}}_d$: desired position, velocity, acceleration. K_p, K_v: diagonal gain matrices.

Why this works: a spring and a damper

Suppose the inner loops deliver exactly the acceleration we ask for, $\ddot{\boldsymbol{p}} = \boldsymbol{a}_d$. Then the position error $\boldsymbol{e}_p = \boldsymbol{p}_d - \boldsymbol{p}$ changes as

$$\ddot{\boldsymbol{e}}_p = \ddot{\boldsymbol{p}}_d - \ddot{\boldsymbol{p}} = \ddot{\boldsymbol{p}}_d - \boldsymbol{a}_d = -K_p\boldsymbol{e}_p - K_v\dot{\boldsymbol{e}}_p \quad \Longrightarrow \quad \ddot{\boldsymbol{e}}_p + K_v\dot{\boldsymbol{e}}_p + K_p\boldsymbol{e}_p = \boldsymbol{0}.$$

This is the equation of a mass on a spring (stiffness k_p) with a damper (friction k_v), one per axis. Its solutions behave like e^{st} with $s^2 + k_v s + k_p = 0$. If $k_p > 0$ and $k_v > 0$, both roots have negative real part, so the error always dies out. Comparing with the standard form $s^2 + 2\zeta\omega_n s + \omega_n^2$:

$$\omega_n = \sqrt{k_p} \text{ (how fast)}, \qquad \zeta = \frac{k_v}{2\sqrt{k_p}} \text{ (how damped)}, \qquad t_{\text{settle}} \approx \frac{4}{\zeta\omega_n}.$$

axis	k_p	k_v	ω_n [rad/s]	ζ	t_{settle} [s]
$x,\ y$	1.5	1.2	1.225	0.490	6.7
z	2.0	1.5	1.414	0.530	5.3

Our `controller.yaml` gains are gentle: $\zeta \approx 0.5$ means a small overshoot, and the error settles in about 6 s. That suits a camera drone that should move smoothly. The term $\ddot{\boldsymbol{p}}_d$ ("feed-forward") supplies the acceleration a perfect pilot would need to follow a moving target; the feedback terms only fix what is left over.

Toy example — 50 cm too low and already climbing

Setpoint $\boldsymbol{p}_d = [0, 0, 2]$ m (the `controller.yaml` hover point), $\boldsymbol{v}_d = \ddot{\boldsymbol{p}}_d = \boldsymbol{0}$. The estimator says $\hat{\boldsymbol{p}} = [0.1,\ -0.2,\ 1.5]$ m and $\hat{\boldsymbol{v}} = [0, 0, 0.3]$ m/s.

$$\boldsymbol{e}_p = \begin{bmatrix} 0 - 0.1 \\ 0 - (-0.2) \\ 2 - 1.5 \end{bmatrix} = \begin{bmatrix} -0.1 \\ 0.2 \\ 0.5 \end{bmatrix}, \quad \boldsymbol{e}_v = \begin{bmatrix} 0 \\ 0 \\ -0.3 \end{bmatrix}, \quad \boldsymbol{a}_d = \begin{bmatrix} 1.5(-0.1) \\ 1.5(0.2) \\ 2(0.5) + 1.5(-0.3) \end{bmatrix} = \begin{bmatrix} -0.15 \\ 0.30 \\ 0.55 \end{bmatrix} \text{ m/s}^2.$$

It is 50 cm low, so it wants to accelerate up. Because it is already climbing at 0.3 m/s, it asks for less: 0.55 instead of 1.0. That is the damper at work, preventing overshoot.

Toy model: `toy_model3/M1_3_SO3_Control_Closed_Loop.mlx`, section "Block 5"

```
p_hat = [0.1; -0.2; 1.5];   v_hat = [0; 0; 0.3];
p_d   = [0; 0; 2];          v_d = [0; 0; 0];     a_ff = [0; 0; 0];
e_p = p_d - p_hat
e_v = v_d - v_hat
a_d = a_ff + P.Kp*e_p + P.Kv*e_v
```

How this controls the real drone

Equation (15) is `compute_position_errors()` and equation (16) is the first part of `compute_desired_force()` in our controller (`a_cmd` is $\boldsymbol{a}_d$):

drones_controller/so3_controller.py — equation (15)

```
32      def compute_position_errors(self, p, v, desired):
33          p_d = desired["p_d"]
34          v_d = desired["v_d"]
35          e_p = p_d - p
36          e_v = v_d - v
37          return e_p, e_v
```

drones_controller/so3_controller.py — equation (16)

```
64          a_cmd = (
65              self.Kp @ e_p
66              + self.Kv @ e_v
67              + a_d
68          )
```

The desired state comes from `WaypointGenerator`: a fixed point $[0, 0, 2]$ with $\boldsymbol{v}_d = \boldsymbol{a}_d = \mathbf{0}$, read from the YAML.

4.6 Block 6 — Force and attitude: which way to lean

The idea in plain words

A quadrotor can only push along its own up-axis. To get the acceleration $\boldsymbol{a}_d$ it must (1) work out the total force needed, including holding up its own weight, and (2) tilt so that its up-axis points along that force. Like a person pushing a heavy cart: to push forward, you lean forward.

Equations (17)–(19): desired force, thrust direction and attitude

$$\boldsymbol{F}_d = m\,(\boldsymbol{a}_d - \boldsymbol{g}) \tag{17}$$

$$\boldsymbol{b}_{3d} = \frac{\boldsymbol{F}_d}{\|\boldsymbol{F}_d\|} \tag{18}$$

$$\boldsymbol{b}_{1c} = [\cos\psi_d,\ \sin\psi_d,\ 0]^\top, \quad \boldsymbol{b}_{2d} = \frac{\boldsymbol{b}_{3d} \times \boldsymbol{b}_{1c}}{\|\boldsymbol{b}_{3d} \times \boldsymbol{b}_{1c}\|}, \quad \boldsymbol{b}_{1d} = \boldsymbol{b}_{2d} \times \boldsymbol{b}_{3d}, \quad R_d = [\boldsymbol{b}_{1d}\ \ \boldsymbol{b}_{2d}\ \ \boldsymbol{b}_{3d}] \tag{19}$$

Derivation

(17) We want $\dot{\boldsymbol{v}} = \boldsymbol{a}_d$. Equation (2) says $\dot{\boldsymbol{v}} = \boldsymbol{g} + \frac{T}{m} R\boldsymbol{e}_3$, so the thrust vector must be $T\,R\boldsymbol{e}_3 = m(\boldsymbol{a}_d - \boldsymbol{g})$. Call this $\boldsymbol{F}_d$. Because $\boldsymbol{g}$ points down, $-\boldsymbol{g}$ points up: even for $\boldsymbol{a}_d = 0$ the drone must push up with $mg_0 = 14.7\,\mathrm{N}$.

(18) $TR\boldsymbol{e}_3 = T\boldsymbol{b}_3$ is a vector of length T along the drone's up-axis. For it to equal $\boldsymbol{F}_d$, the up-axis must point along $\boldsymbol{F}_d$, so $\boldsymbol{b}_{3d} = \boldsymbol{F}_d/\|\boldsymbol{F}_d\|$, and the thrust must be $T = \|\boldsymbol{F}_d\|$. This is the key fact of quadrotor flight: *position is controlled through the attitude.*

(19) $\boldsymbol{b}_{3d}$ fixes the tilt (two of the three rotation directions). The third, yaw (where the nose points),

is our free choice ψ_d. We build the remaining two axes so that they are unit length and at right angles:

- $\boldsymbol{b}_{1c}$ is the heading we would like, flat on the ground.
- $\boldsymbol{b}_{2d} = \boldsymbol{b}_{3d} \times \boldsymbol{b}_{1c}$ (normalised) is at right angles to both the up-axis and the heading: the drone's "left".
- $\boldsymbol{b}_{1d} = \boldsymbol{b}_{2d} \times \boldsymbol{b}_{3d}$ is at right angles to both: the nose, as close to the wanted heading as the tilt allows.

The three columns are orthonormal and right-handed, so $R_d^\top R_d = I$ and $\det R_d = 1$: R_d is a true rotation. The construction only fails if $\boldsymbol{b}_{3d}$ is parallel to $\boldsymbol{b}_{1c}$ (a 90° tilt); the code then nudges the yaw.

Toy example — continuing the example of block 5

$\boldsymbol{a}_d = [-0.15, 0.30, 0.55]$, $m = 1.5$, $\boldsymbol{g} = [0, 0, -9.81]$, $\psi_d = 0$:

$$\boldsymbol{F}_d = 1.5\,[-0.15,\ 0.30,\ 0.55 + 9.81] = [-0.225,\ 0.450,\ 15.540]\ \text{N}, \qquad \|\boldsymbol{F}_d\| = 15.548\ \text{N},$$

$$\boldsymbol{b}_{3d} = [-0.0145,\ 0.0289,\ 0.9995], \qquad R_d = \begin{bmatrix} 0.9999 & 0.0000 & -0.0145 \\ 0.0004 & 0.9996 & 0.0289 \\ 0.0145 & -0.0289 & 0.9995 \end{bmatrix}.$$

Tilt $= \arccos(0.9995) = 1.85°$: lean very slightly towards $-x$ and $+y$ (where the target is) and push 15.5 N, more than the 14.7 N weight because the drone must also climb. Check: $\|R_d^\top R_d - I\| = 1.1 \times 10^{-16}$.

Toy model: `toy_model3/M1_3_SO3_Control_Closed_Loop.mlx`, section "Block 6"

```
F_d = P.m*(a_d - P.g)                  % N
b3d = F_d/norm(F_d);
b1c = [cos(psi_d); sin(psi_d); 0];
b2d = cross(b3d, b1c)/norm(cross(b3d, b1c));
b1d = cross(b2d, b3d);
R_d = [b1d b2d b3d]
```

How this controls the real drone

Equation (17) is the end of `compute_desired_force()`. The code writes $\boldsymbol{F}_d = m(\boldsymbol{a}_{\text{cmd}} + g\boldsymbol{e}_3)$, which is the same thing because $-\boldsymbol{g} = g\boldsymbol{e}_3$ in ENU:

drones_controller/so3_controller.py — equation (17)

```
70          F_d = self.mass * (
71              a_cmd + self.gravity * self.e3
72          )
73          return F_d
```

Equations (18)–(19) are `compute_desired_attitude()`, including the safety nudge of the yaw:

drones_controller/so3_controller.py — equations (18)–(19)

```
82          norm_F = np.linalg.norm(F_d)
83          if norm_F < 1e-6:
84              return np.eye(3)
85          b3 = F_d / norm_F
86          yaw_heading = np.array([
```

```
            np.cos(yaw_d),
            np.sin(yaw_d),
            0.0
        ])
        b2 = np.cross(b3, yaw_heading)
        norm_b2 = np.linalg.norm(b2)
        if norm_b2 < 1e-6:
            yaw_d += 1e-3
            yaw_heading = np.array([
                np.cos(yaw_d),
                np.sin(yaw_d),
                0.0
            ])
            b2 = np.cross(b3, yaw_heading)
            norm_b2 = np.linalg.norm(b2)
        b2 /= norm_b2
        b1 = np.cross(b2, b3)
        b1 /= np.linalg.norm(b1)
        R_d = np.column_stack([
            b1,
            b2,
            b3
        ])
        return R_d
```

The node converts R_d to a quaternion and publishes it on the topic `/drones_controller/desired_attitude`, so we can watch in RViz which way the controller wants to lean.

4.7 Block 7 — Attitude control on SO(3) (inner loop)

The idea in plain words

"I should be leaning like R_d, but I am leaning like $\hat{R}$. Which way do I twist, and how hard?" The inner loop computes an attitude error directly from the two rotation matrices (no angles, so no gimbal lock), a spin error, and a torque that removes both, like a spring and a damper for rotation. It runs faster than the position loop, because the drone must tilt before it can move.

Equations (20)–(23): geometric attitude controller

$$\boldsymbol{e}_R = \tfrac{1}{2}\left(R_d^\top \hat{R} - \hat{R}^\top R_d\right)^\vee \tag{20}$$

$$\boldsymbol{e}_\omega = \hat{\boldsymbol{\omega}} - \hat{R}^\top R_d\, \boldsymbol{\omega}_d \tag{21}$$

$$\boldsymbol{\tau} = -K_R \boldsymbol{e}_R - K_\omega \boldsymbol{e}_\omega + \hat{\boldsymbol{\omega}} \times J\hat{\boldsymbol{\omega}} - J\left([\hat{\boldsymbol{\omega}}]_\times \hat{R}^\top R_d \boldsymbol{\omega}_d - \hat{R}^\top R_d \dot{\boldsymbol{\omega}}_d\right) \tag{22}$$

$$T = \boldsymbol{F}_d \cdot \hat{R}\, \boldsymbol{e}_3 \tag{23}$$

(20) The attitude error

We need a number that says how far apart two attitudes are. Use

$$\Psi = \tfrac{1}{2} \operatorname{tr}\left(I - R_d^\top \hat{R}\right) = 1 - \cos\varphi,$$

where φ is the angle of the rotation that takes R_d to $\hat{R}$ (because the trace of a rotation by φ is $1 + 2\cos\varphi$). $\Psi = 0$ when the attitudes agree and $\Psi = 2$ when they are upside down to each other. Now turn the drone a tiny bit in the direction $\boldsymbol{\eta}$ and see how Ψ changes. A short calculation gives

$$\text{change of } \Psi = \boldsymbol{e}_R \cdot \boldsymbol{\eta}, \qquad \boldsymbol{e}_R = \tfrac{1}{2}(R_d^\top \hat{R} - \hat{R}^\top R_d)^\vee.$$

So $\boldsymbol{e}_R$ is the *slope* (gradient) of the error: it points in the direction of turning that increases the error fastest. The controller twists the opposite way, $-K_R\boldsymbol{e}_R$, "downhill". For a single-axis error of angle φ about the axis $\boldsymbol{n}$, $\boldsymbol{e}_R = \sin\varphi\,\boldsymbol{n} \approx \varphi\,\boldsymbol{n}$ for small angles.

(21) The angular-velocity error

The relative rotation $R_e = R_d^\top \hat{R}$ changes as $\dot{R}_e = R_e[\boldsymbol{e}_\omega]_\times$ with $\boldsymbol{e}_\omega = \hat{\boldsymbol{\omega}} - \hat{R}^\top R_d \boldsymbol{\omega}_d$ (differentiate and use (3) for both). So $\boldsymbol{e}_\omega$ is how fast the drone turns *relative to how it should be turning*, in body axes. $\hat{R}^\top R_d\boldsymbol{\omega}_d$ is simply the desired spin written in the drone's own axes. Note the order: *actual minus desired*.

(22) The torque, and why it is guaranteed to work

Differentiate $\boldsymbol{e}_\omega$ and replace $J\dot{\hat{\boldsymbol{\omega}}}$ using (4):

$$J\dot{\boldsymbol{e}}_\omega = \boldsymbol{\tau} - \hat{\boldsymbol{\omega}} \times J\hat{\boldsymbol{\omega}} + J\big([\hat{\boldsymbol{\omega}}]_\times \hat{R}^\top R_d\boldsymbol{\omega}_d - \hat{R}^\top R_d\dot{\boldsymbol{\omega}}_d\big).$$

Now insert the torque (22). The gyroscopic term and the bracket cancel exactly, and what remains is

$$J\dot{\boldsymbol{e}}_\omega = -K_R\boldsymbol{e}_R - K_\omega\boldsymbol{e}_\omega,$$

a rotational spring (K_R) and damper (K_ω). To prove it always settles, take (with $K_R = k_R I$, $K_\omega = k_\omega I$) the "energy"

$$V = \underbrace{\tfrac{1}{2}\boldsymbol{e}_\omega^\top J\boldsymbol{e}_\omega}_{\text{spin energy}} + \underbrace{k_R\Psi}_{\text{spring energy}}.$$

Its rate of change is $\dot{V} = \boldsymbol{e}_\omega^\top(-k_R\boldsymbol{e}_R - k_\omega\boldsymbol{e}_\omega) + k_R\,\boldsymbol{e}_R \cdot \boldsymbol{e}_\omega = -k_\omega\|\boldsymbol{e}_\omega\|^2 \le 0$. The energy can only go down, like a pendulum with friction, so the drone settles at R_d from *any* starting attitude except exactly upside down. This kind of proof is called a **Lyapunov** argument.

(23) The thrust

Thrust can only act along the *current* up-axis $\hat{R}\boldsymbol{e}_3$. The useful part of $\boldsymbol{F}_d$ along that axis is the dot product $T = \boldsymbol{F}_d \cdot \hat{R}\boldsymbol{e}_3$. When the attitude is right, $T = \|\boldsymbol{F}_d\|$. While the drone is still tilted the wrong way, T is automatically smaller, so it does not push hard in a wrong direction.

Toy example — recovering from a 10 degree roll

The drone is rolled 10° and still rolling further at 0.2 rad/s; it should be level ($R_d = I$, $\boldsymbol{\omega}_d = \dot{\boldsymbol{\omega}}_d = 0$), $\boldsymbol{F}_d = [0, 0, 14.715]$:

$$\boldsymbol{e}_R = \tfrac{1}{2}(\hat{R} - \hat{R}^\top)^\vee = [\sin 10^\circ,\ 0,\ 0] = [0.1736,\ 0,\ 0], \qquad \boldsymbol{e}_\omega = [0.2,\ 0,\ 0],$$

$$\tau_x = -4(0.1736) - 0.8(0.2) + 0 = -0.8546 \text{ N m}, \qquad T = 14.715\cos 10^\circ = 14.491 \text{ N}.$$

A torque about $-x$ rolls it back, and the thrust is reduced to the part that is still vertical.

Recovery from 60°. The toy model starts the drone tilted 60° and lets block 7 bring it back. After 3 s the error is 2×10^{-6} degrees, and the energy V goes down all the time (fig. 7).

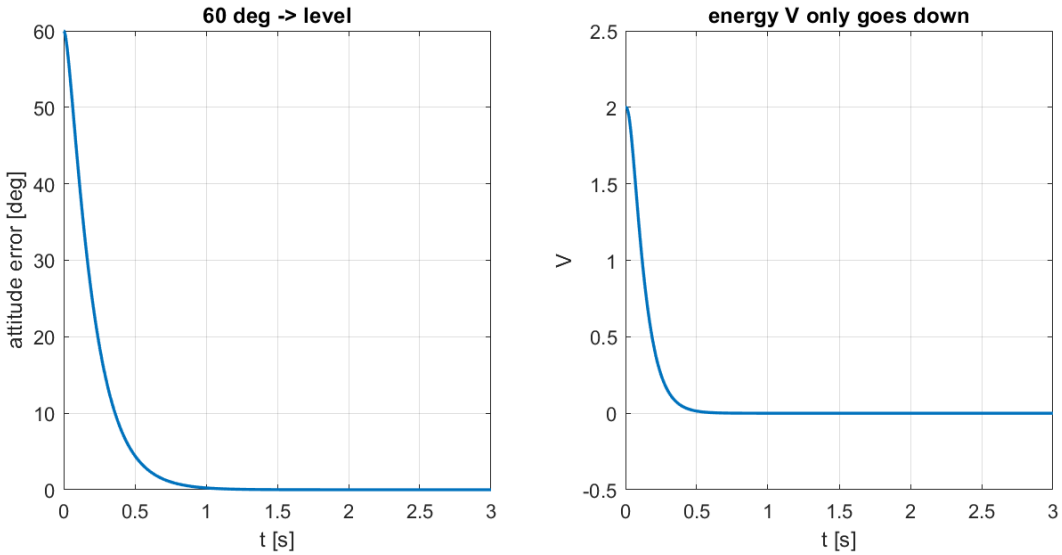


Figure 7: Block 7 alone: from 60° to level. Right: the energy V only goes down, as the proof promised.

Toy model: `toy_model3/M1_3_SO3_Control_Closed_Loop.mlx`, section "Block 7"

```
e_R = 0.5*DroneLib.vee(Rd'*R_hat - R_hat'*Rd)
e_w = w_hat - R_hat'*Rd*w_d
tau = -P.KR*e_R - P.Kw*e_w + cross(w_hat, P.J*w_hat) ...
      - P.J*(DroneLib.hat(w_hat)*R_hat'*Rd*w_d - R_hat'*Rd*wdot_d)
T = dot(Fd, R_hat*P.e3)
```

How this controls the real drone

Equations (20), (21) and (22) are three functions of our controller, written exactly as above:

drones_controller/so3_controller.py — equation (20)

```
134         E = (
135             R_d.T @ R
136             -
137             R.T @ R_d
138         )
139         e_R = 0.5 * vee(E)
140         return e_R
```

drones_controller/so3_controller.py — equation (21)

```
152         e_omega = (
153             omega
154             -
155             R.T @ R_d @ omega_d
156         )
157         return e_omega
```

drones_controller/so3_controller.py — equation (22)

```
181         relative_R = R.T @ R_d
182         coriolis = np.cross(
183             omega,
184             self.J @ omega
185         )
```

```
        omega_hat = np.array([
            [0.0, -omega[2], omega[1]],
            [omega[2], 0.0, -omega[0]],
            [-omega[1], omega[0], 0.0]
        ])
        feedforward = self.J @ (
            omega_hat @ relative_R @ omega_d
            -
            relative_R @ omega_dot_d
        )
        M_d = (
            -self.KR @ e_R
            -
            self.KOmega @ e_omega
            +
            coriolis
            -
            feedforward
        )
        return M_d
```

Equation (23) is not in the Python code, because our node does not send thrust to ArduPilot (see section 5). The toy model computes it.

4.8 Block 8 — Four motor speeds (control allocation)

The idea in plain words

The drone does not have a "torque knob". It has four motors. Spinning the right-side motors faster rolls it left; spinning the front ones faster pitches it back; spinning the clockwise pair faster yaws it. Block 8 solves the reverse problem: which four rotor thrusts give exactly the wanted T and $\boldsymbol{\tau}$?

Equations (24)–(25): control allocation and motor speeds

$$\begin{bmatrix} T \\ \boldsymbol{\tau} \end{bmatrix} = \Gamma \, \boldsymbol{f}, \qquad \Gamma = \begin{bmatrix} 1 & 1 & 1 & 1 \\ y_1 & y_2 & y_3 & y_4 \\ -x_1 & -x_2 & -x_3 & -x_4 \\ \kappa_1 c_m & \kappa_2 c_m & \kappa_3 c_m & \kappa_4 c_m \end{bmatrix} \tag{24}$$

$$\Omega_i = \sqrt{\frac{(\Gamma^{-1}\boldsymbol{u})_i}{k_f}} \quad \text{clipped to } [\Omega_{\min}, \Omega_{\max}] \tag{25}$$

f_i: thrust of rotor i; (x_i, y_i): its position on the frame; $\kappa_i = \pm 1$: its spin direction; c_m: yaw torque per newton; k_f: thrust constant ($f_i = k_f \Omega_i^2$).

Derivation

Each rotor i pushes up with f_i at the point $\boldsymbol{r}_i = [x_i, y_i, 0]$ of the frame.

- Total thrust: the pushes add up, $T = f_1 + f_2 + f_3 + f_4$ (row 1 of Γ).

- Roll and pitch torque: a force at a distance creates a torque $\boldsymbol{r}_i \times f_i \boldsymbol{e}_3 = f_i[y_i, -x_i, 0]$ (rows 2 and 3). A rotor on the right ($y_i < 0$) creates negative roll torque.
- Yaw torque: air resists each spinning propeller, so each motor twists the body the opposite way with a torque proportional to its thrust, $\kappa_i c_m f_i$ (row 4).

A propeller's thrust grows with the square of its speed: $f_i = k_f \Omega_i^2$. Inverting both relations gives (25). Motors cannot spin backwards or faster than their maximum, so the speeds are clipped.

For the Iris (ArduPilot "Quad-X": 1 front-right CCW, 2 back-left CCW, 3 front-left CW, 4 back-right CW):

$$\Gamma = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -0.22 & 0.22 & 0.22 & -0.22 \\ -0.13 & 0.13 & -0.13 & 0.13 \\ -0.016 & -0.016 & 0.016 & 0.016 \end{bmatrix}.$$

Toy example — hover, and the roll correction of block 7

Hover $\boldsymbol{u} = [14.715, 0, 0, 0]$: by symmetry every rotor gives $f_i = 14.715/4 = 3.679\,\text{N}$, so $\Omega_i = \sqrt{3.679/8.549 \times 10^{-6}} = 656\,\text{rad/s}$ (about 6260 rpm).

Roll correction $\boldsymbol{u} = [14.491, -0.8546, 0, 0]$: by symmetry $f_1 = f_4 = a$ and $f_2 = f_3 = b$. Row 1: $2a + 2b = 14.491$. Row 2: $-0.22a + 0.22b + 0.22b - 0.22a = 0.44(b - a) = -0.8546$. Solving: $a = 4.594\,\text{N}$, $b = 2.652\,\text{N}$, so $\boldsymbol{\Omega} = [733,\ 557,\ 557,\ 733]\,\text{rad/s}$. The right-side rotors (1 and 4) speed up, the left ones slow down, and the drone rolls back to level.

Toy model: `toy_model3/M1_3_SO3_Control_Closed_Loop.mlx`, section "Block 8 (DroneLib.mixer)"

```
fr = P.Gamma \ u;                                          % rotor thrusts
fr = min(max(fr, P.kf*P.Omega_min^2), P.kf*P.Omega_max^2); % motor limits
Omega = sqrt(fr/P.kf);                                     % motor speeds
```

How this controls the real drone

This block lives inside ArduPilot (its "motor mixer"); our node never talks to the motors directly. The matrix Γ above is the one ArduPilot uses for a Quad-X frame, scaled by the Iris arm lengths.

4.9 All of Model 1 together: the closed loop

The last section of `M1_3` connects everything as in fig. 1: the true drone (block 1) → noisy gyro and camera → EKF (block 4) → controller on the *estimate* (blocks 5–7) → mixer (block 8) → the true drone, in gusty wind, following a climbing helix for 25 s. After the start-up transient the drone follows the helix with an RMS error of **4.7 cm**, the estimator is accurate to **2.2 cm**, and the motors stay between 649 and 664 rad/s.

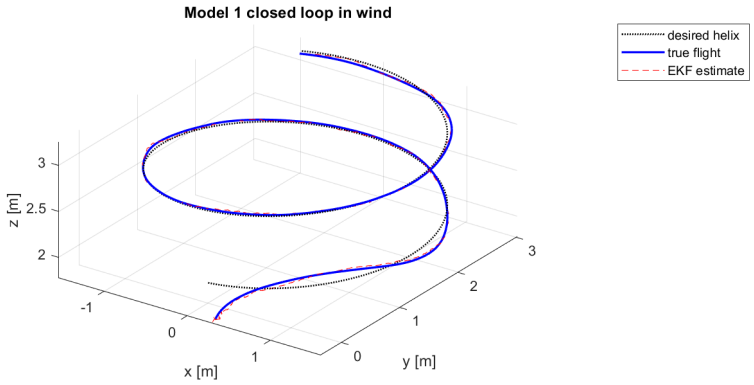


Figure 8: The complete Model 1 loop in wind: desired helix (dotted), true flight (blue), EKF estimate (red, dashed).

Toy model: `toy_model3/M1_3_SO3_Control_Closed_Loop.mlx`, section "The closed loop"

```
ref = DroneLib.helix(t(k));                                % mission
[ph, vh, Rh, wh] = DroneLib.unpack(xh);                    % the ESTIMATE
u = DroneLib.so3_control(ph, vh, Rh, wh, ref, P);          % blocks 5-7
[Om(:,k), ~, u_app] = DroneLib.mixer(u, P);                % block 8
x  = DroneLib.rk4(x, u_app, dt, P, gust);                  % block 1, the real drone
[xh, Pk] = DroneLib.ekf_predict(xh, Pk, u_app, dt, P, Q);  % block 4
```

Recap of Model 1

#	Block	In	Out	The idea in one sentence
1	Dynamics	thrust, torque	motion	Push along the body's up-axis, gravity pulls down, torque spins the body.
2	Linearization	operating point	A, B	Zoomed in, the drone behaves like a linear system.
3	Error dynamics	A, B, Δt	F	Small errors grow linearly, and fast (t^3), without feedback.
4	EKF (VIO)	commands, sensors	$\hat{\boldsymbol{x}}_k$, P_k	Predict with physics, correct with camera and IMU, weighted by trust.
5	Position control	$\hat{\boldsymbol{p}}, \hat{\boldsymbol{v}}$, target	$\boldsymbol{a}_d$	Pull towards the target like a spring with a damper.
6	Force & attitude	$\boldsymbol{a}_d$, yaw	$\boldsymbol{F}_d$, R_d	To accelerate sideways, lean; thrust direction = force direction.
7	SO(3) attitude	$\hat{R}, \hat{\boldsymbol{\omega}}, R_d, \boldsymbol{F}_d$	$\boldsymbol{\tau}$, T	Twist downhill on the attitude error; the energy always decreases.
8	Allocation	T, $\boldsymbol{\tau}$	$\Omega_1..\Omega_4$	Share thrust and torque among four rotors; speed $\propto \sqrt{\text{thrust}}$.

5 How Our ROS 2 Node Actually Controls the Drone

Section 4 explained each equation. This section follows our real program, `drones_controller`, from start-up through one complete control step, line by line, with numbers. At the end you should be able to point at any line of `controller_node.py` or `so3_controller.py` and say which equation it is and why it is there.

5.1 Start-up: reading the parameters and building the controller

When the node starts, it declares every parameter with a default value. ROS 2 then replaces the defaults with the values in `controller.yaml`. For example:

controller_node.py — some of the parameter declarations

```
        self.declare_parameter(
            "gravity",
            9.81
        )
        self.declare_parameter(
            "kp",
            [1.5, 1.5, 2.0]
        )
        self.declare_parameter(
            "kv",
            [1.2, 1.2, 1.5]
        )
        self.declare_parameter(
            "kR",
            [4.0, 4.0, 2.0]
        )
        self.declare_parameter(
            "kOmega",
            [0.8, 0.8, 0.5]
        )
        self.declare_parameter(
            "inertia",
            [0.02, 0.02, 0.04]
        )
```

Then it creates the two objects that do the work: the *trajectory generator* (where should I be?) and the *SO(3) controller* (how do I get there?). The gains are turned into diagonal matrices K_p, K_v, K_R, K_ω and the inertia into J:

controller_node.py — the two working objects

```
        self.trajectory = WaypointGenerator(
            desired_position,
            desired_yaw
        )
        self.controller = SO3Controller(
            mass=mass,
            gravity=gravity,
            inertia=inertia,
            kp=kp,
            kv=kv,
            kR=kR,
            kOmega=kOmega
        )
```

so3_controller.py — ___init___(): gains become diagonal matrices

```
        self.mass = float(mass)
        self.gravity = float(gravity)
        self.J = np.diag(np.array(inertia, dtype=float))
        self.Kp = np.diag(np.array(kp, dtype=float))
        self.Kv = np.diag(np.array(kv, dtype=float))
        self.KR = np.diag(np.array(kR, dtype=float))
```

```
        self.KOmega = np.diag(np.array(kOmega, dtype=float))
        self.e3 = np.array([0.0, 0.0, 1.0])
```

Next it subscribes to the estimator's output and starts a timer that calls `control_loop()` at `control_rate` = 25 times per second (one call every 40 ms):

controller_node.py — subscribing to the state estimate

```
        self.pose_sub = self.create_subscription(
            PoseStamped,
            pose_topic,
            self.pose_callback,
            state_qos
        )
        self.twist_sub = self.create_subscription(
            TwistStamped,
            twist_topic,
            self.twist_callback,
            state_qos
        )
```

controller_node.py — the control timer

```
        period = 1.0 / self.control_rate
        self.timer = self.create_timer(
            period,
            self.control_loop
        )
```

Small inconsistency to know about

The default `control_rate` in the code is 50 Hz, but `controller.yaml` sets 25 Hz. When the node is launched with the YAML file the rate is 25 Hz. If it is started without the YAML, it runs at 50 Hz. The YAML value is the one we use everywhere in this document.

5.2 Between control steps: receiving the state (block 4's output)

Whenever ArduPilot publishes a new estimate, a *callback* stores it. `pose_callback` stores $\boldsymbol{p}$ and turns the quaternion into R (shown in section 4.1). `twist_callback` stores $\boldsymbol{v}$ and $\boldsymbol{\omega}$:

controller_node.py — twist_callback()

```
    def twist_callback(self, msg):
        self.velocity = np.array([
            msg.twist.linear.x,
            msg.twist.linear.y,
            msg.twist.linear.z
        ])
        self.omega = np.array([
            msg.twist.angular.x,
            msg.twist.angular.y,
            msg.twist.angular.z
        ])
        self.last_twist_time = self.get_clock().now()
```

Both remember *when* the data arrived, so that the control step can refuse to use old data.

5.3 One control step, line by line

Every 40 ms the timer calls `control_loop()`:

controller_node.py — control_loop(), first part

```
526     def control_loop(self):
527         if not self.state_is_valid():
528             return
529         desired = (
530             self.trajectory
531             .get_desired_state()
532         )
533         result = self.controller.compute(
534             p=self.position,
535             v=self.velocity,
536             R=self.rotation,
537             omega=self.omega,
538             desired=desired
539         )
```

1. `state_is_valid()`: is every part of the state present, younger than 0.5 s and a finite number? If not, *do nothing* this step (safety).
2. `get_desired_state()`: the mission. Today it is a fixed waypoint, so $\boldsymbol{p}_d = [0, 0, 2]$, $\boldsymbol{v}_d = \boldsymbol{a}_d = \mathbf{0}$, $\psi_d = 0$, $\boldsymbol{\omega}_d = \dot{\boldsymbol{\omega}}_d = \mathbf{0}$:

trajectory_generator.py — get_desired_state()

```
11     def get_desired_state(self):
12         p_d = self.position.copy()
13         v_d = np.zeros(3)
14         a_d = np.zeros(3)
15         omega_d = np.zeros(3)
16         omega_dot_d = np.zeros(3)
17         return {
18             "p_d": p_d,
19             "v_d": v_d,
20             "a_d": a_d,
21             "yaw_d": self.yaw,
22             "omega_d": omega_d,
23             "omega_dot_d": omega_dot_d
```

3. `controller.compute()`: blocks 5, 6 and 7, in that order:

so3_controller.py — compute(): blocks 5 → 6 → 7

```
221         e_p, e_v = self.compute_position_errors(
222             p,
223             v,
224             desired
225         )
226         F_d = self.compute_desired_force(
227             e_p,
228             e_v,
229             desired["a_d"]
230         )
231         R_d = self.compute_desired_attitude(
232             F_d,
233             desired["yaw_d"]
```

```
        )
        e_R = self.compute_attitude_error(
            R,
            R_d
        )
        e_omega = self.compute_angular_velocity_error(
            R,
            R_d,
            omega,
            desired["omega_d"]
        )
        M_d = self.compute_moment(
            R,
            R_d,
            omega,
            desired["omega_d"],
            desired["omega_dot_d"],
            e_R,
            e_omega
        )
```

4. Publish every intermediate result on `/drones_controller/...`, so we can plot them while flying.
5. If `enable_command` is true, send a velocity command (next subsection).

Toy example — one complete control step of our node, with real numbers

The estimate says the drone is at $\boldsymbol{p} = [0.1, -0.2, 1.5]\,\text{m}$, climbing at $\boldsymbol{v} = [0, 0, 0.3]\,\text{m/s}$, level ($R = I$) and not turning ($\boldsymbol{\omega} = \mathbf{0}$). The waypoint is $[0, 0, 2]$. Running the real `SO3Controller.compute()` on these numbers gives:

Output	Equation	Value	Meaning
`e_p`	(15)	$[-0.1,\ 0.2,\ 0.5]\,\text{m}$	target is 10 cm west, 20 cm north, 50 cm up
`e_v`	(15)	$[0,\ 0,\ -0.3]\,\text{m/s}$	already climbing a bit
`F_d`	(16)–(17)	$[-0.225,\ 0.450,\ 15.540]\,\text{N}$	push mostly up, slightly towards the target
`R_d`	(18)–(19)	tilt of 1.85°	lean slightly towards $-x$, $+y$
`e_R`	(20)	$[0.0289,\ 0.0145,\ -0.0002]$	the drone is level but should lean
`e_omega`	(21)	$[0,\ 0,\ 0]$	no spin, none wanted
`M_d`	(22)	$[-0.1158,\ -0.0579,\ 0.0004]\,\text{N m}$	twist towards the lean

Check the torque: $-K_R \boldsymbol{e}_R = -[4(0.0289),\ 4(0.0145),\ 2(-0.0002)] = [-0.1158,\ -0.0579,\ 0.0004]$, and $\boldsymbol{e}_\omega = 0$ adds nothing. A torque about $-x$ tips the top of the drone towards $+y$, and a torque about $-y$ tips it towards $-x$: exactly the direction of $\boldsymbol{F}_d$. Every number agrees with the toy model (sections 4.5 to 4.7).

5.4 What is actually sent to the drone: the velocity command

ArduPilot's ROS 2 interface `/ap/v1/cmd_vel` accepts a *velocity*, not a force and torque. So our node does not send $\boldsymbol{F}_d$ or $\boldsymbol{\tau}$ (they are published only for monitoring). Instead `publish_velocity_command()` turns the position and velocity errors into a velocity request:

controller_node.py — publish_velocity_command(): the velocity law

```
        kp_cmd = np.array([
            0.8,
            0.8,
            0.8
        ])
        kv_cmd = np.array([
            0.2,
            0.2,
            0.2
        ])
        velocity_command = (
            kp_cmd * e_p
            +
            kv_cmd * e_v
        )
        speed = np.linalg.norm(
            velocity_command
        )
        if speed > self.max_velocity:
            velocity_command *= (
                self.max_velocity / speed
            )
```

$$\boldsymbol{v}_{\text{cmd}} = 0.8\,\boldsymbol{e}_p + 0.2\,\boldsymbol{e}_v, \qquad \text{and if } \|\boldsymbol{v}_{\text{cmd}}\| > 1 \text{ m/s, scale it down to length 1 m/s.}$$

This is again a spring (0.8: "go towards the target, faster when further away") and a damper (0.2: "slow down if you are already moving"). The scaling keeps the *direction* and enforces `max_velocity`.

Toy example — the velocity command

Close to the target (same state as above): $\boldsymbol{v}_{\text{cmd}} = 0.8[-0.1, 0.2, 0.5] + 0.2[0, 0, -0.3] = [-0.08,\ 0.16,\ 0.34]$ m/s, length 0.38 m/s, below the limit, sent as it is.

Far from the target: drone at $[3, -2, 0.5]$, at rest. $\boldsymbol{e}_p = [-3, 2, 1.5]$, so $\boldsymbol{v}_{\text{cmd}} = [-2.4,\ 1.6,\ 1.2]$, length 3.12 m/s, too fast. Scaled by $1/3.12$: $[-0.768,\ 0.512,\ 0.384]$ m/s, length exactly 1 m/s, still pointing straight at the target.

ArduPilot then does the rest. Inside the flight controller it runs its own version of blocks 5–8: a velocity controller that asks for an acceleration, which becomes a lean angle, which an attitude controller turns into torques, which the mixer turns into motor speeds. So the whole chain of section 4 still happens; our node does the outer "where should I go" part and ArduPilot the inner, fast part.

Two things to check in the simulator

- **The sign of z.** The last lines send `linear.z = -velocity_command[2]`:

controller_node.py — filling the message

```
        msg.twist.linear.x = float(
            velocity_command[0]
        )
        msg.twist.linear.y = float(
            velocity_command[1]
        )
        msg.twist.linear.z = float(
            -velocity_command[2]
        )
```

The pose and twist that we read are in ENU (z up), and the header says `"map"`. If ArduPilot reads this command in ENU too, the minus sign makes the drone go *down* when it should go up. It is only right if the interface expects z pointing down (NED). This must be checked once in SITL: command a small climb and watch the altitude.

- **Safety switch.** `enable_command` is `false` in the YAML, so by default the node only computes and publishes; nothing moves until we deliberately switch it on.

5.5 Summary: from equation to line of code

Equation	Block	Where in our code
(1)–(4)	dynamics	Gazebo (not in our code); result read in `pose_callback`, `twist_callback`
R from quaternion	–	`state_adapter.py`: `quaternion_to_rotation_matrix()`, lines 4–42
(5)–(14)	EKF (VIO)	ArduPilot EKF3; freshness check in `state_is_valid()`, lines 447–496
(15)	position errors	`so3_controller.py`: `compute_position_errors()`, lines 32–40
(16)–(17)	desired force	`so3_controller.py`: `compute_desired_force()`, lines 42–74
(18)–(19)	desired attitude	`so3_controller.py`: `compute_desired_attitude()`, lines 76–126
(20)	attitude error	`so3_controller.py`: `compute_attitude_error()`, lines 128–142
(21)	rate error	`so3_controller.py`: `compute_angular_velocity_error()`, lines 144–158
(22)	torque	`so3_controller.py`: `compute_moment()`, lines 160–210
(23)–(25)	thrust, mixer	ArduPilot (our node sends a velocity instead)
$\boldsymbol{v}_{\text{cmd}}$	command	`controller_node.py`: `publish_velocity_command()`, lines 644–734

6 Model 2: Learning from Data and Optimal Control

Model 1 needs the physics: the mass m, the inertia J, the rotor constants. Model 2 asks a different question: *if we only had recorded flight data, could we still build a good controller?* It learns a linear model from data (blocks 1–2) and then computes the best controller for that model in two ways (fig. 3):

- **LQR** (blocks 3–6): the mathematically best *linear* controller. Very cheap to run, but it knows nothing about limits.
- **MPC** (blocks 7–9): plans the next second optimally *and* respects limits such as "never faster than 1 m/s", at the cost of solving a small optimisation problem every step.

Block 10 flies the real, nonlinear drone with the result. Block 11 is an **extended Kalman filter** that works out the drone's state from camera pixels and the gyroscope, using the learned model. That state is what the controllers of blocks 9 and 10 need, so block 11 closes the loop. All Model 2 matrices are in *discrete time* with $\Delta t = 0.04\,\text{s}$ (25 Hz, the `control_rate`): $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$. Toy model files:

- `M2_1_Flight_Data_DMDc.mlx` (blocks 1–2),
- `M2_2_LQR_Bellman_Riccati_DARE.mlx` (blocks 3–6),
- `M2_3_MPC_QP_Apply_to_Drone.mlx` (blocks 7–10),

- `M2_4_EKF_State_Estimation.mlx` (block 11).

6.1 Block 1 — Recording flight data

The idea in plain words

Fly the drone and write down, 25 times per second, where it was and what we told it to do. To make the data useful we measure everything as a *difference from hover*, and we deliberately give the drone small random pushes so that the data contains enough variety to learn from.

Equation (M2.1): sampled state and input, as deviations from hover

$$\boldsymbol{x}_k = \begin{bmatrix} \boldsymbol{p}(t_k) - \boldsymbol{p}_{\text{ref}} \\ \boldsymbol{v}(t_k) \\ \operatorname{Log}(R(t_k)) \\ \boldsymbol{\omega}(t_k) \end{bmatrix} \in \mathbb{R}^{12}, \qquad \boldsymbol{u}_k = \begin{bmatrix} T(t_k) - mg_0 \\ \boldsymbol{\tau}(t_k) \end{bmatrix} \in \mathbb{R}^4, \qquad t_k = k\,\Delta t. \tag{M2.1}$$

Why it is written this way

Differences from hover. Hover is a balance point of (1)–(4): level, still, thrust = weight, every rate is zero. Near it, the linear model $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$ has *no constant term*. If we used the raw thrust T instead of $T - mg_0$, gravity would appear as a constant $+mg_0$ that this form cannot represent, and the fit would be poor.

Attitude as three numbers. The learning method needs plain vectors, so the rotation matrix is replaced by its rotation vector $\operatorname{Log}(R)$ (section 2). Near hover these three numbers are roll, pitch and yaw in radians.

A pilot plus random pushes. A free drone is unstable (section 4.3), so the data must be recorded with some stabilising pilot flying it (the role ArduPilot plays). But if the input were *only* feedback, $\boldsymbol{u} = -K_0\boldsymbol{x}$, every input would be a fixed combination of the states, and the computer could not tell "what the drone does" apart from "what the pilot does". Adding small random pushes (± 2 N of thrust, ± 0.03 N m of torque, each held for 0.2 s) fixes this. This is called **persistent excitation**.

Toy example — the recorded data set

Five flights of 25 s each at 25 Hz, starting from random points near $[0, 0, 2]$ m: 5×625 samples of $\boldsymbol{x}_k \in \mathbb{R}^{12}$ and $\boldsymbol{u}_k \in \mathbb{R}^4$, with a little sensor noise. Flights 1–4 are used to learn, flight 5 is kept back to test. The largest tilt in all the data is 6.5°, well inside the range where the drone is nearly linear (section 4.2).

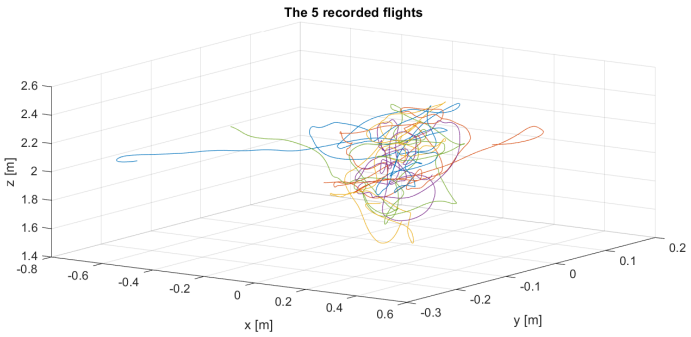


Figure 9: The five recorded flights around the hover point.

Toy model: `toy_model3/M2_1_Flight_Data_DMDc.mlx`, section "Block 1"

```
Xr(:,k) = DroneLib.to_dev(x, P.p_hover) + noise;        % logged state x_k
if mod(k-1, 5) == 0                                       % new random push
    exc = [2; 0.03; 0.03; 0.01].*(2*rand(4,1) - 1);       %   every 0.2 s
end
[~, ~, u_app] = DroneLib.mixer(u_trim + DroneLib.pilot(Xr(:,k), P) + exc, P);
Ur(:,k) = u_app - u_trim;                                 % logged input u_k
```

How this controls the real drone

In the project, block 1 is a ROS 2 bag recording of `/ap/v1/pose/filtered`, `/ap/v1/twist/filtered` and the commands we send, from SITL or a real flight (`ros2 bag record ...`). The pose and twist are exactly the inputs our node already reads, so no new sensor is needed, and nothing about mass or inertia needs to be known.

6.2 Block 2 — DMDc: learning A and B from the data

The idea in plain words

Look at thousands of moments of the form "the drone was *here*, we pushed like *this*, and 40 ms later it was *there*", and find the single linear rule $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$ that explains all of them best. It is curve fitting for motion. It is the EDMD idea from the course notes (`K = Psi2*pinv(Psi1)`) with the input added; the name DMDc means *Dynamic Mode Decomposition with control*.

Equations (M2.2)–(M2.4): snapshot matrices and the least-squares model

$$X = [\boldsymbol{x}_1 \ \cdots \ \boldsymbol{x}_{N-1}], \quad X^+ = [\boldsymbol{x}_2 \ \cdots \ \boldsymbol{x}_N], \quad U = [\boldsymbol{u}_1 \ \cdots \ \boldsymbol{u}_{N-1}] \tag{M2.2}$$

$$[\,A \ \ B\,] = X^+ \begin{bmatrix} X \\ U \end{bmatrix}^{\dagger} \tag{M2.3}$$

$$\boldsymbol{x}_{k+1} \approx A\,\boldsymbol{x}_k + B\,\boldsymbol{u}_k \tag{M2.4}$$

Derivation

Put the states side by side in a matrix X, the same states shifted one step later in X^+, and the inputs in U. We want one matrix $G = [A \ \ B]$ such that, for every column,

$$\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k = G \begin{bmatrix} \boldsymbol{x}_k \\ \boldsymbol{u}_k \end{bmatrix}.$$

Written for all columns at once:

$$X^+ \approx G\,\Omega, \qquad \Omega = \begin{bmatrix} X \\ U \end{bmatrix}.$$

With noisy data this cannot hold exactly, so we choose the G with the smallest total squared prediction error $\|X^+ - G\Omega\|^2$ (least squares, section 2.10). Setting the derivative with respect to G to zero:

$$-2\,(X^+ - G\Omega)\,\Omega^\top = 0 \quad \Longrightarrow \quad G\,\Omega\Omega^\top = X^+\Omega^\top \quad \Longrightarrow \quad G = X^+\Omega^\top(\Omega\Omega^\top)^{-1} = X^+\Omega^\dagger.$$

$\Omega\Omega^\top$ is a 16×16 matrix. It can only be inverted if Ω has 16 independent rows ($12 + 4$), which is exactly what the random pushes of block 1 guarantee. The first 12 columns of G are A and the last 4 are B.

What should the answer be? If the data came from the physics of Model 1, the exact answer is the discrete version of the Jacobians (6a): $A = e^{A_c \Delta t}$ and $B = \int_0^{\Delta t} e^{A_c s} ds\, B_c$ (the "zero-order hold": the input is held constant for one step). So we can check DMDc against the physics.

Toy example — DMDc by hand, one number

The true system is $x_{k+1} = 0.9x_k + 0.5u_k$, but we pretend not to know 0.9 and 0.5. Three recorded steps: $X = [1,\ 1.4,\ 0.76]$, $U = [1,\ -1,\ 0.5]$, $X^+ = [1.4,\ 0.76,\ 0.934]$. Then

$$\Omega\Omega^\top = \begin{bmatrix} 1 + 1.96 + 0.5776 & 1 - 1.4 + 0.38 \\ 1 - 1.4 + 0.38 & 1 + 1 + 0.25 \end{bmatrix} = \begin{bmatrix} 3.5376 & -0.02 \\ -0.02 & 2.25 \end{bmatrix},$$

$$X^+\Omega^\top = \begin{bmatrix} 1.4 + 1.064 + 0.70984 & 1.4 - 0.76 + 0.467 \end{bmatrix} = \begin{bmatrix} 3.17384 & 1.107 \end{bmatrix},$$

$$[a \quad b] = \frac{1}{3.5376 \cdot 2.25 - 0.02^2} \begin{bmatrix} 3.17384 & 1.107 \end{bmatrix} \begin{bmatrix} 2.25 & 0.02 \\ 0.02 & 3.5376 \end{bmatrix} = \begin{bmatrix} 0.9000 & 0.5000 \end{bmatrix}.$$

The hidden numbers are found exactly, because this data has no noise.

The drone (2500 snapshots, rank of $\Omega = 16$):

entry	physical meaning	learned by DMDc	physics
$A_{1,4}$	Δt (position from velocity)	0.0399	0.0400
$A_{4,8}$	$g_0 \Delta t$ (v_x from pitch)	0.3909	0.3924
$A_{5,7}$	$-g_0 \Delta t$ (v_y from roll)	−0.3862	−0.3924
$B_{6,1}$	$\Delta t/m$ (v_z from thrust)	0.0268	0.0267
$B_{10,2}$	$\Delta t/J_x$ (roll rate from roll torque)	2.0007	2.0000
$B_{12,4}$	$\Delta t/J_z$ (yaw rate from yaw torque)	1.0096	1.0000

Overall the learned A is within 2.6% of the physics and B within 1.1%. The data even "knows" the mass: $\Delta t/B_{6,1} = 1.494\,\text{kg}$ (true: 1.5 kg). On flight 5, which was never used for learning, predicting 1 s ahead from the inputs alone gives a position error of 1.33 cm, against 1.26 cm for the exact physics model.

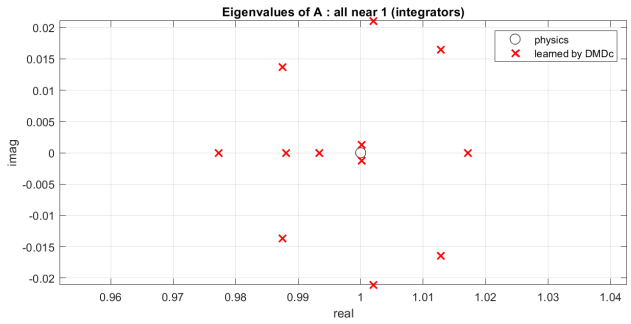


Figure 10: Eigenvalues of the learned A (crosses) and of the physics (circles). All lie at or very near 1: a chain of integrators, exactly as block 3 of Model 1 predicted.

Toy model: `toy_model3/M2_1_Flight_Data_DMDc.mlx`, section "Block 2"

```
Omega = [Xa; Ua];
rank_Omega = rank(Omega)                % must be 12 + 4 = 16 (enough excitation)
```

```
G = Xpa*pinv(Omega);                    % (M2.3)
A = G(:, 1:12);
B = G(:, 13:16);
```

How this controls the real drone

DMDc runs offline, on a laptop, on a recorded bag. If the drone changes (a heavier camera, a new battery, a different frame), a two-minute flight is enough to learn the new A and B, without measuring anything by hand.

6.3 Block 3 — The Bellman equation: what is "the best" controller?

The idea in plain words

Before we can find the best controller we must say what "best" means. We give the flight a *score*: every moment we are off target costs points, and every push of the motors costs points. The best controller is the one with the lowest total score. Bellman's idea: the best plan from here is the best *first move* plus the best plan from wherever that move takes us.

Equations (M2.5)–(M2.7): cost, Bellman equation, quadratic value function

$$J = \sum_{k=0}^{\infty} \left(\boldsymbol{x}_k^\top Q\, \boldsymbol{x}_k + \boldsymbol{u}_k^\top R\, \boldsymbol{u}_k \right) \tag{M2.5}$$

$$V(\boldsymbol{x}_k) = \min_{\boldsymbol{u}_k} \left[\boldsymbol{x}_k^\top Q \boldsymbol{x}_k + \boldsymbol{u}_k^\top R \boldsymbol{u}_k + V(\boldsymbol{x}_{k+1}) \right], \qquad \boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k \tag{M2.6}$$

$$V(\boldsymbol{x}) = \boldsymbol{x}^\top P\, \boldsymbol{x} \tag{M2.7}$$

(M2.5) The score

$\boldsymbol{x}^\top Q \boldsymbol{x}$ is a weighted sum of squared errors and $\boldsymbol{u}^\top R \boldsymbol{u}$ a weighted sum of squared pushes. Big Q: "I hate being off target". Big R: "I hate using the motors". We choose them with **Bryson's rule**: $Q_{ii} = 1/x_{i,\max}^2$ and $R_{jj} = 1/u_{j,\max}^2$, where $x_{i,\max}$ is the largest error we are willing to accept:

quantity	largest acceptable value	weight
position error	0.5 m	$1/0.5^2 = 4$
velocity	1 m/s (`max_velocity`)	1
roll, pitch / yaw	0.2 rad / 0.5 rad	25 / 4
angular rates	1 rad/s	1
extra thrust	5 N	0.04
roll, pitch torque / yaw torque	0.2 N m / 0.05 N m	25 / 400

So a 50 cm position error "hurts" as much as flying at 1 m/s, tilting 0.2 rad, or using 5 N of extra thrust.

(M2.6) Principle of optimality

Split the infinite sum into the first step and all the rest:

$$V(\boldsymbol{x}_k) = \min_{\boldsymbol{u}_k, \boldsymbol{u}_{k+1}, \ldots} \Big[\ell(\boldsymbol{x}_k, \boldsymbol{u}_k) + \sum_{j>k} \ell(\boldsymbol{x}_j, \boldsymbol{u}_j) \Big] = \min_{\boldsymbol{u}_k} \Big[\ell(\boldsymbol{x}_k, \boldsymbol{u}_k) + \underbrace{\min_{\boldsymbol{u}_{k+1}, \ldots} \sum_{j>k} \ell(\boldsymbol{x}_j, \boldsymbol{u}_j)}_{V(\boldsymbol{x}_{k+1})} \Big],$$

where ℓ is the score of one step. Whatever the first move is, the remaining moves must be the best ones for the place it leads to; otherwise we could improve them and lower the total. A problem with infinitely many unknowns becomes a problem with *one* unknown, $\boldsymbol{u}_k$.

(M2.7) The value function is a bowl

Guess that the lowest possible score from $\boldsymbol{x}$ is a quadratic, $V(\boldsymbol{x}) = \boldsymbol{x}^\top P \boldsymbol{x}$. Then the bracket in (M2.6) is a quadratic in $\boldsymbol{u}_k$. Its minimum (block 4) is linear in $\boldsymbol{x}_k$, and putting it back gives again a quadratic in $\boldsymbol{x}_k$ (block 5). The guess is self-consistent, so all that is left to find is the matrix P.

Toy example — one Bellman step by hand

System $x_{k+1} = 1.1x_k + 0.5u_k$ (on its own it grows 10% per step: unstable). $q = r = 1$, the next value is $V(x) = x^2$, and we are at $x = 1$:

$$\text{score}(u) = \underbrace{1}_{qx^2} + \underbrace{u^2}_{ru^2} + \underbrace{(1.1 + 0.5u)^2}_{V(x_{k+1})}, \qquad \frac{d}{du} = 2u + (1.1 + 0.5u) = 0 \;\Rightarrow\; u^* = -\frac{1.1}{2.5} = -0.44,$$

$$V(1) = 1 + 0.1936 + (1.1 - 0.22)^2 = 1.968.$$

MATLAB's `fminsearch` on the same function returns $u^* = -0.4400$ and 1.9680.

Toy model: `toy_model3/M2_2_LQR_Bellman_Riccati_DARE.mlx`, section "Block 3"

```
a = 1.1;  b = 0.5;  q = 1;  r = 1;
cost = @(u) q*1^2 + r*u.^2 + 1*(a*1 + b*u).^2;
u_star = -a*b/(r + b^2)                  % by hand
[u_search, V1] = fminsearch(cost, 0)   % by the inbuilt optimiser
```

6.4 Block 4 — The Linear Quadratic Regulator (LQR)

The idea in plain words

Do the minimisation of block 3 once, in general, with matrices. The answer is beautifully simple: the best push is a fixed matrix K times the current error. The whole controller is one 4×12 table of numbers.

Equations (M2.8)–(M2.9): optimal gain and control law

$$K = (R + B^\top P B)^{-1} B^\top P A \tag{M2.8}$$

$$\boldsymbol{u}_k = -K\, \boldsymbol{x}_k \tag{M2.9}$$

Derivation

Put $V(\boldsymbol{x}_{k+1}) = (A\boldsymbol{x} + B\boldsymbol{u})^\top P(A\boldsymbol{x} + B\boldsymbol{u})$ into the bracket of (M2.6). It is a bowl in $\boldsymbol{u}$ (section 2.8). Its slope with respect to $\boldsymbol{u}$, using $\partial(\boldsymbol{u}^\top M\boldsymbol{u})/\partial\boldsymbol{u} = 2M\boldsymbol{u}$, must be zero:

$$2R\boldsymbol{u} + 2B^\top P(A\boldsymbol{x} + B\boldsymbol{u}) = \mathbf{0} \quad \Longrightarrow \quad (R + B^\top PB)\,\boldsymbol{u} = -B^\top PA\,\boldsymbol{x}.$$

$R + B^\top PB$ is positive definite (because R is), so it can be inverted and the point really is the bottom of the bowl. Hence $\boldsymbol{u}^* = -(R + B^\top PB)^{-1}B^\top PA\,\boldsymbol{x} = -K\boldsymbol{x}$.

Toy example — the scalar system, and the drone's gain

With the final $P = 3.1215$ (from block 6): $K = \dfrac{ab\,P}{r + b^2P} = \dfrac{1.1 \cdot 0.5 \cdot 3.1215}{1 + 0.25 \cdot 3.1215} = 0.9643$. With control, $x_{k+1} = (a - bK)x_k = 0.6179\,x_k$. Without control the state grew 10% per step; with LQR it *shrinks* 38% per step (section 2.9).

The drone's K, two of its 48 numbers (from block 6):

$$\delta T = -9.04\,\delta z - 6.92\,v_z, \qquad \tau_x = 0.273\,\delta y + 0.321\,v_y - 1.444\,\theta_x - 0.276\,\omega_x.$$

Read them: too high or rising ⇒ less thrust. Too far north ($\delta y > 0$) ⇒ positive roll torque, which tips the drone towards $-y$, back to the target; but less so if it is already rolled (θ_x) or rolling (ω_x).

6.5 Block 5 — The Riccati recursion: finding P step by step

The idea in plain words

K needs P, and P is "the lowest score from here". Start with a controller that looks 0 steps ahead, then 1 step, then 2, and so on. Each time, the plan sees one more step of the future. After enough steps, looking further does not change anything: we have the infinite-horizon P.

Equation (M2.10): Riccati difference equation

$$P_{k+1} = A^\top P_k A - A^\top P_k B(R + B^\top P_k B)^{-1}B^\top P_k A + Q, \qquad P_0 = Q \tag{M2.10}$$

(k counts "steps of look-ahead", not time.)

Derivation

Put the best push $\boldsymbol{u}^* = -K\boldsymbol{x}$ back into the bracket of (M2.6):

$$V_{\text{new}}(\boldsymbol{x}) = \boldsymbol{x}^\top\big[Q + K^\top RK + (A - BK)^\top P(A - BK)\big]\boldsymbol{x}.$$

Multiply out $(A - BK)^\top P(A - BK) = A^\top PA - A^\top PBK - K^\top B^\top PA + K^\top B^\top PBK$ and add $K^\top RK$. From (M2.8), $(R + B^\top PB)K = B^\top PA$, so $K^\top(R + B^\top PB)K = K^\top B^\top PA$, and two terms cancel:

$$V_{\text{new}} = \boldsymbol{x}^\top\big[Q + A^\top PA - A^\top PBK\big]\boldsymbol{x} = \boldsymbol{x}^\top\big[Q + A^\top PA - A^\top PB(R + B^\top PB)^{-1}B^\top PA\big]\boldsymbol{x}.$$

The matrix in brackets is the new P: that is (M2.10).

Toy example — the scalar recursion: $a = 1.1$, $b = 0.5$, $q = r = 1$, $P_0 = 1$

$$P_{k+1} = 1 + 1.21P_k - \frac{(0.55P_k)^2}{1 + 0.25P_k}$$

k	0	1	2	3	4	5	6	7	8	∞
P_k	1	1.968	2.596	2.905	3.036	3.089	3.109	3.117	3.120	3.1215

$P_1 = 1.968$ is exactly the Bellman step of block 3. For the drone, starting from $P_0 = Q$, the change becomes smaller than 10^{-12} after 236 iterations (fig. 11): looking more than about 10 s ahead changes nothing.

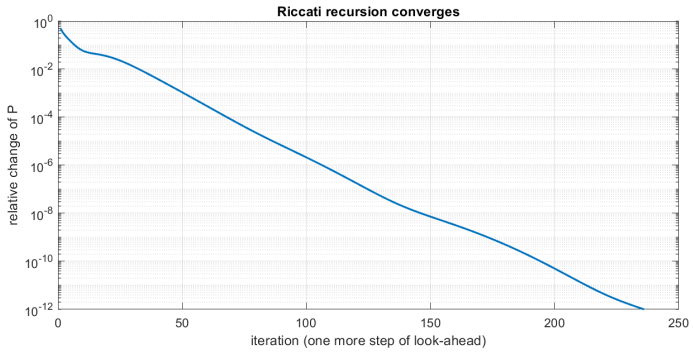


Figure 11: Riccati recursion for the drone: the change of P falls steadily until it has converged.

Toy model: `toy_model3/M2_2_LQR_Bellman_Riccati_DARE.mlx`, section "Riccati recursion"

```
Pk = Q;
for it = 1:5000
    Kk = (R + B'*Pk*B) \ (B'*Pk*A);                    % (M2.8)
    Pn = A'*Pk*A - A'*Pk*B*Kk + Q;                     % (M2.10)
    change(it) = norm(Pn - Pk, 'fro')/norm(Pn, 'fro');
    Pk = Pn;
    if change(it) < 1e-12, break, end
end
```

6.6 Block 6 — The DARE: jumping straight to the answer

The idea in plain words

When the recursion has stopped changing, $P_{k+1} = P_k$. Writing that down gives one equation for the final P, the *Discrete Algebraic Riccati Equation*. It can be solved directly instead of iterating. Its solution gives the final gain K and also tells us the total score in advance.

Equation (M2.11): the Discrete Algebraic Riccati Equation

$$P = A^\top PA - A^\top PB(R + B^\top PB)^{-1}B^\top PA + Q \qquad \text{(M2.11)}$$

Two useful facts

The total score is known in advance. Rearranged, (M2.11) says $P = Q + K^\top RK + (A - BK)^\top P(A - BK)$. Along the controlled flight $\boldsymbol{x}_{k+1} = (A - BK)\boldsymbol{x}_k$ this means $\boldsymbol{x}_k^\top P \boldsymbol{x}_k - \boldsymbol{x}_{k+1}^\top P \boldsymbol{x}_{k+1} = \boldsymbol{x}_k^\top Q \boldsymbol{x}_k + \boldsymbol{u}_k^\top R \boldsymbol{u}_k$. Adding this up over all steps, everything in between cancels (like $(a-b)+(b-c)+(c-d) = a-d$) and, because $\boldsymbol{x}_k \to 0$:

$$J^* = \boldsymbol{x}_0^\top P \, \boldsymbol{x}_0.$$

Solving without a toolbox. The toy model solves (M2.11) with a standard trick (stable eigenvectors of a 24×24 matrix pencil, `DroneLib.dare`), so no Control System Toolbox is needed.

Toy example — the scalar DARE is a quadratic equation

$P = 1 + 1.21P - \dfrac{0.3025P^2}{1 + 0.25P}$. Multiply by $(1 + 0.25P)$ and collect terms: $0.25P^2 - 0.46P - 1 = 0$, so

$$P = \frac{0.46 + \sqrt{0.46^2 + 1}}{0.5} = 3.1215$$

(the positive root), exactly the limit of the recursion.

The drone. The DARE solution satisfies (M2.11) to 6.5×10^{-14} and agrees with the 236-step recursion to 1.6×10^{-11}. The learned drone on its own has spectral radius $\rho(A) = 1.017 > 1$ (unstable); with LQR $\rho(A - BK) = 0.950 < 1$ (stable). Starting 50/30/20 cm away, the simulated total score is 41.4373, and the prediction $\boldsymbol{x}_0^\top P \boldsymbol{x}_0$ is also 41.4373.

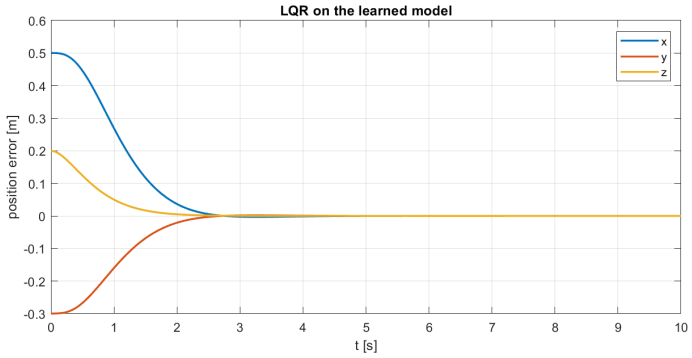


Figure 12: LQR on the learned model: the position errors die out smoothly.

Toy model: `toy_model3/M2_2_LQR_Bellman_Riccati_DARE.mlx`, section "DARE solved directly"

```
P = DroneLib.dare(A, B, Q, R);                    % (M2.11)
K = (R + B'*P*B) \ (B'*P*A);                      % (M2.8)
spectral_radius_closed = max(abs(eig(A - B*K)))   % < 1 : LQR makes it stable
```

How this controls the real drone — LQR

On the drone, LQR would be one line every 40 ms: $\boldsymbol{u} = \boldsymbol{u}_{\text{trim}} - K\boldsymbol{x}$, with $\boldsymbol{x}$ from the state estimate. That costs 48 multiplications, nothing for any flight computer. Its weakness: it knows nothing about limits. Our node's velocity law $\boldsymbol{v}_{\text{cmd}} = 0.8\boldsymbol{e}_p + 0.2\boldsymbol{e}_v$ (section 5) is a hand-tuned gain of exactly this kind, and it needs a separate clip to respect `max_velocity`.

6.7 Block 7 — Model Predictive Control (MPC): planning with limits

The idea in plain words

MPC is like a chess player. Every 40 ms it imagines the next second of flight, finds the best sequence of moves that never breaks a rule ("never faster than 1 m/s", "never ask a motor for more than it can give"), plays only the first move, and then thinks again with the new position.

Equation (M2.12): the finite-horizon problem with limits

$$\begin{aligned} \min_{\boldsymbol{u}_0,\dots,\boldsymbol{u}_{N-1}} \quad & \sum_{k=0}^{N-1} \left(\boldsymbol{x}_k^\top Q \boldsymbol{x}_k + \boldsymbol{u}_k^\top R \boldsymbol{u}_k\right) + \boldsymbol{x}_N^\top P_f \, \boldsymbol{x}_N \\ \text{subject to} \quad & \boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k, \quad \boldsymbol{x}_0 = \hat{\boldsymbol{x}}_k \text{ (the estimate of block 11)}, \\ & \boldsymbol{u}_{\min} \le \boldsymbol{u}_k \le \boldsymbol{u}_{\max}, \qquad |C\boldsymbol{x}_k| \le \boldsymbol{x}_{\max} \end{aligned} \tag{M2.12}$$

Reasoning

Real drones have limits: total thrust between about 5 and 25 N, the torque the rotors can produce, `max_velocity` = 1 m/s, and a maximum tilt for a camera. With limits there is no neat formula like LQR's any more, so MPC solves the problem numerically, and to keep it small:

- it only plans N steps ahead ($N = 25$: one second);
- the infinite rest of the flight is replaced by a final cost $\boldsymbol{x}_N^\top P_f \boldsymbol{x}_N$.

Why $P_f = P$ from the DARE. We showed that $\boldsymbol{x}_N^\top P \boldsymbol{x}_N$ is *exactly* the best score of the rest of the flight when no limit is active. So with this choice, when no limit is touched, MPC gives exactly the LQR answer. That is the link between the two branches of fig. 3.

The drone's limits in the toy model: $5 \le T \le 25$ N (so $\delta T \in [-9.715, 10.285]$ N), $|\tau_{x,y}| \le 0.3$ N m, $|\tau_z| \le 0.05$ N m, $|v_i| \le 1$ m/s, roll and pitch ≤ 0.35 rad (20°). C simply picks these five states out of $\boldsymbol{x}$.

6.8 Block 8 — Turning MPC into a Quadratic Program (QP)

The idea in plain words

Write every future state in terms of the unknown future pushes. Then the whole problem becomes: "find the lowest point of a bowl, but stay inside a fence". That is called a *quadratic program*, and computers solve it reliably in milliseconds.

Equations (M2.13)–(M2.15): stacked prediction, cost and constraints

$$\mathbf{X} = S_x \boldsymbol{x}_0 + S_u \mathbf{U}, \qquad S_x = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix}, \quad S_u = \begin{bmatrix} B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ A^{N-1}B & \cdots & AB & B \end{bmatrix} \tag{M2.13}$$

$$\min_{\mathbf{U}} \ \tfrac{1}{2}\mathbf{U}^\top H \mathbf{U} + \boldsymbol{f}^\top \mathbf{U}, \qquad H = 2(S_u^\top \bar{Q} S_u + \bar{R}), \quad \boldsymbol{f} = 2 S_u^\top \bar{Q} S_x \boldsymbol{x}_0 \tag{M2.14}$$

$$\text{subject to} \quad G\mathbf{U} \le \boldsymbol{h}, \qquad A_{eq}\mathbf{U} = \boldsymbol{b}_{eq} \tag{M2.15}$$

$\mathbf{X} = [\boldsymbol{x}_1; \ldots; \boldsymbol{x}_N]$, $\mathbf{U} = [\boldsymbol{u}_0; \ldots; \boldsymbol{u}_{N-1}]$, $\bar{Q} = \mathrm{diag}(Q, \ldots, Q, P_f)$, $\bar{R} = \mathrm{diag}(R, \ldots, R)$.

Derivation

(M2.13) Prediction. Apply the model again and again: $\boldsymbol{x}_1 = A\boldsymbol{x}_0 + B\boldsymbol{u}_0$; $\boldsymbol{x}_2 = A\boldsymbol{x}_1 + B\boldsymbol{u}_1 = A^2\boldsymbol{x}_0 + AB\boldsymbol{u}_0 + B\boldsymbol{u}_1$; and in general $\boldsymbol{x}_k = A^k\boldsymbol{x}_0 + \sum_{j=0}^{k-1} A^{k-1-j}B\boldsymbol{u}_j$. Stacking $k = 1..N$ gives (M2.13). The future is *linear* in the unknown pushes.

(M2.14) Cost. The score is $\boldsymbol{x}_0^\top Q\boldsymbol{x}_0 + \mathbf{X}^\top\bar{Q}\mathbf{X} + \mathbf{U}^\top\bar{R}\mathbf{U}$. Substitute $\mathbf{X}$ and multiply out:

$$\mathbf{U}^\top(S_u^\top\bar{Q}S_u + \bar{R})\mathbf{U} + 2\,\boldsymbol{x}_0^\top S_x^\top\bar{Q}S_u\,\mathbf{U} + \underbrace{\boldsymbol{x}_0^\top(Q + S_x^\top\bar{Q}S_x)\boldsymbol{x}_0}_{\text{does not depend on } \mathbf{U}}.$$

Drop the last part (it does not change where the minimum is) and write the rest as $\frac{1}{2}\mathbf{U}^\top H\mathbf{U} + \boldsymbol{f}^\top\mathbf{U}$. H is positive definite because $\bar{R}$ is, so the bowl has *one* bottom and no false minima.

(M2.15) Constraints. Input limits: $\mathbf{U} \le \mathbf{U}_{\max}$ and $-\mathbf{U} \le -\mathbf{U}_{\min}$. State limits $|\bar{C}\mathbf{X}| \le \bar{\boldsymbol{x}}_{\max}$ become, after substituting $\mathbf{X}$, $\bar{C}S_u\mathbf{U} \le \bar{\boldsymbol{x}}_{\max} - \bar{C}S_x\boldsymbol{x}_0$ and $-\bar{C}S_u\mathbf{U} \le \bar{\boldsymbol{x}}_{\max} + \bar{C}S_x\boldsymbol{x}_0$. Stacked, these give $G\mathbf{U} \le \boldsymbol{h}$. Equalities $A_{eq}\mathbf{U} = \boldsymbol{b}_{eq}$ can force, for example, "be exactly at the target after N steps".

The solver. The toy model includes its own QP solver (`DroneLib.qp`, an interior-point method), so no Optimization Toolbox is needed. For the drone it needs about 10 iterations.

Toy example — a two-step QP by hand: $a = 1.1$, $b = 0.5$, $q = r = 1$, $N = 2$, $P_f = 3.1215$, $x_0 = 1$

$$S_x = \begin{bmatrix} 1.1 \\ 1.21 \end{bmatrix}, \quad S_u = \begin{bmatrix} 0.5 & 0 \\ 0.55 & 0.5 \end{bmatrix}, \quad \bar{Q} = \mathrm{diag}(1,\ 3.1215), \quad \bar{R} = I,$$

$$H = \begin{bmatrix} 4.3885 & 1.7168 \\ 1.7168 & 3.5607 \end{bmatrix}, \qquad \boldsymbol{f} = \begin{bmatrix} 5.2547 \\ 3.7770 \end{bmatrix}.$$

- **No limits:** $\mathbf{U}^* = -H^{-1}\boldsymbol{f} = [-0.9643,\ -0.5958]$. The first move equals the LQR move $-Kx_0 = -0.9643$ *exactly*, as promised by $P_f = P$.
- **With $|u_k| \le 0.5$:** $\mathbf{U}^* = [-0.5,\ -0.5]$. Both moves sit on the fence.
- **Forcing $x_2 = 0$** with $[0.55\ \ 0.5]\,\mathbf{U} = -1.21$: $\mathbf{U}^* = [-1.3057,\ -0.9837]$. Check: $1.21 + 0.55(-1.3057) + 0.5(-0.9837) = 0$.

The drone ($N = 25$): 100 unknowns and 450 inequalities. Starting 5 cm off, no limit is touched and MPC gives $[-0.4557, -0.0138, -0.0137, -0.0004]$, the same as $-K\boldsymbol{x}_0$ to every printed digit. Starting 2 m too low, LQR would ask for +18.1 N of extra thrust, far more than the allowed +10.285 N. MPC uses exactly +10.285 N and plans a climb that touches, but never exceeds, 1 m/s.

Toy model: `toy_model3/M2_3_MPC_QP_Apply_to_Drone.mlx`, section "Block 8, toy example"

```
Sx = [a; a^2];
Su = [b 0; a*b b];
Qbar = diag([1 Pf]);  Rbar = eye(2);
H = 2*(Su'*Qbar*Su + Rbar)                      % (M2.14)
f = 2*Su'*Qbar*Sx*x0
U_free    = (-H\f)'                             % no limits
U_limited = DroneLib.qp(H, f, [eye(2); -eye(2)], 0.5*ones(4,1))'   % |u| <= 0.5
```

6.9 Block 9 — Receding horizon: plan, act a little, plan again

Equation (M2.16): receding-horizon control law

$$\mathbf{U}^*(\boldsymbol{x}_k) = [\boldsymbol{u}_0^*,\ \boldsymbol{u}_1^*,\ \ldots,\ \boldsymbol{u}_{N-1}^*] \quad \Longrightarrow \quad \text{apply only } \boldsymbol{u}_0^*,\ \text{measure } \boldsymbol{x}_{k+1},\ \text{solve again} \quad \text{(M2.16)}$$

The idea in plain words

The plan is perfect only for the model, and the model is never perfect (it was learned, and the wind is not in it). So we never follow a whole plan. We use its first move for 40 ms, measure where we really are, and plan again. This turns a plan into feedback: mistakes are corrected 25 times per second instead of piling up.

Toy example — 8 s on the learned model, MPC against LQR

Start 1.5 m, 1.0 m and 1.2 m away from the target, at rest. MPC never exceeds 1 m/s (largest speed 1.000 m/s). LQR, even when its pushes are clipped to the same motor limits, reaches 1.172 m/s, because clipping the push does not limit the *speed*. The median time to solve one QP was 2.6 ms, well inside the 40 ms budget.

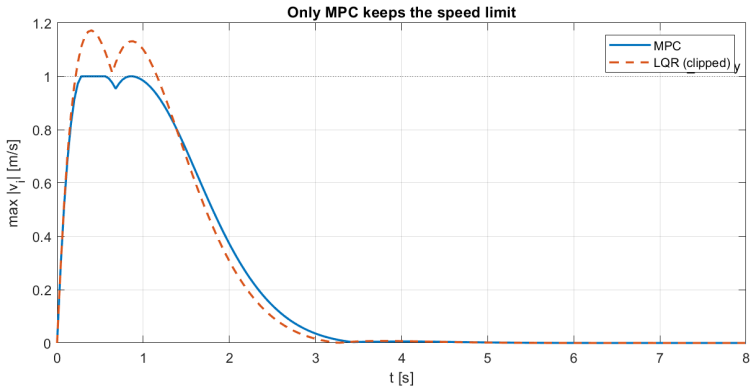


Figure 13: Largest speed component over time: only MPC respects `max_velocity` = 1 m/s.

Toy model: `toy_model3/M2_3_MPC_QP_Apply_to_Drone.mlx`, section "Block 9"

```
for k = 1:Kn
    um = DroneLib.mpc_solve(mpc, Xm(:,k));            % solve the QP, keep u_0*
    ul = min(max(-K*Xl(:,k), u_min), u_max);          % LQR, clipped
    Xm(:,k+1) = A*Xm(:,k) + B*um;
    Xl(:,k+1) = A*Xl(:,k) + B*ul;
end
```

6.10 Block 10 — Applying it to the drone

Equations (M2.17)–(M2.18): from optimal input to motor speeds

$$\boldsymbol{u}_k = \boldsymbol{u}_{\text{trim}} + \begin{cases} \boldsymbol{u}_0^*(\boldsymbol{x}_k) & \text{(MPC)} \\ -K\boldsymbol{x}_k & \text{(LQR)} \end{cases} = \begin{bmatrix} T_k \\ \boldsymbol{\tau}_k \end{bmatrix}, \qquad \boldsymbol{u}_{\text{trim}} = \begin{bmatrix} mg_0 \\ \mathbf{0} \end{bmatrix} \tag{M2.17}$$

$$\boldsymbol{f} = \Gamma^{-1}\boldsymbol{u}_k, \qquad \Omega_i = \sqrt{f_i/k_f} \qquad \text{(the mixer of block 8 of Model 1)} \tag{M2.18}$$

Reasoning

The controllers of Model 2 work with *differences from hover*. The motors need real values, so the hover thrust mg_0 is added back. The rest is the same motor mixer as Model 1 (section 4.8). The loop closes through block 11: the EKF works out the new state of the *real, nonlinear* drone from its sensors, and block 9 runs again from that estimate. (In the M2_3 flight the controllers are still handed the true state plus a little noise, so that blocks 1–10 can be checked on their own; M2_4 removes that shortcut.)

Toy example — one control sample, then a whole flight

The drone is at $[1.5, -1.0, 0.8]$ m, at rest; the target is $[0, 0, 2]$. MPC gives $\delta\boldsymbol{u} = [10.285, -0.279, -0.300, 0.002]$: full allowed climb thrust and the largest allowed roll and pitch torques. Adding hover: $\boldsymbol{u} = [25.000, -0.279, -0.300, 0.002]$, rotor thrusts $\boldsymbol{f} = [7.115, 5.327, 6.539, 6.019]$ N and motor speeds $\boldsymbol{\Omega} = [912, 789, 875, 839]$ rad/s.

The real test: the nonlinear drone, with gusts and sensor noise, for 10 s, flown by controllers designed only on the model learned from data:

controller	largest speed [m/s]	error after 10 s [m]	motor speeds [rad/s]
MPC	0.994	0.015	584 – 913
LQR (clipped)	1.118	0.031	613 – 911

Both bring the real drone to the target. Only MPC keeps `max_velocity` = 1 m/s.

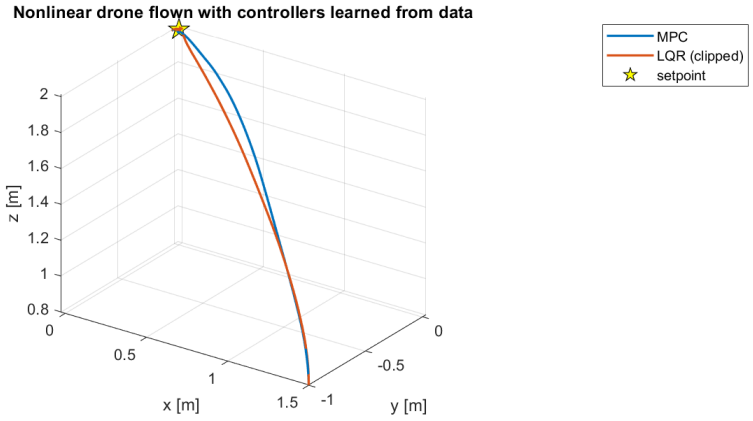


Figure 14: The nonlinear drone flown by MPC and by LQR, both designed only from recorded data.

How this controls the real drone — Model 2

Model 2 is not inside our ROS 2 node today; it is the data-driven alternative we studied and tested in the toy model. It plugs into the same place as the SO(3) controller: it takes the state from `/ap/v1/pose/filtered` and `/ap/v1/twist/filtered` (converted to $\boldsymbol{x}_k$ with (M2.1)) and produces a command. Because ArduPilot accepts velocities, the practical first step would be an MPC whose output is the velocity $\boldsymbol{v}_{\text{cmd}}$ with the limit $\|\boldsymbol{v}\| \leq 1\,\text{m/s}$ written into the problem, instead of the clip in `publish_velocity_command()`.

6.11 Block 11 — Working out the state: an Extended Kalman Filter

Why this block exists, and where it sits

Blocks 9 and 10 need all 12 numbers of $\boldsymbol{x}_k$ every 40 ms. No sensor gives them. The camera gives *pixels*, the gyroscope gives *body rates*, and nothing measures the velocity at all. Up to block 10 the toy model cheated and handed the controllers the true state.

Block 11 is the missing piece. After the drone moves (block 10), the camera and gyro measure it, and the EKF turns those readings into the estimate $\hat{\boldsymbol{x}}_k$. That estimate goes back to MPC (block 9, as its start $\boldsymbol{x}_0$) or to the LQR law $-K\hat{\boldsymbol{x}}_k$. In fig. 3 it sits at the bottom and closes the loop.

It keeps the spirit of Model 2: it *predicts with the learned model* A, B (no mass or inertia needed) and learns its own noise level from the data. It is called *extended* because the camera equation is not a straight line, so it has to be linearized with a Jacobian at every step.

Three letters are already taken in Model 2 (P value matrix, Q weight, K LQR gain). So for the filter we write Σ for the covariance, Q_e for its process noise and L for its gain. Otherwise it is exactly the filter of Model 1, block 4 (section 4.4).

Equations (M2.19)–(M2.24): an EKF on the learned model

Predict with the learned model (it is already linear, so $F = A$):

$$\hat{\boldsymbol{x}}_{k+1} = A\,\hat{\boldsymbol{x}}_k + B\,\boldsymbol{u}_k, \qquad \Sigma_{k+1} = A\,\Sigma_k A^\top + Q_e \tag{M2.19}$$

$$Q_e = \operatorname{cov}(\boldsymbol{x}_{k+1} - A\boldsymbol{x}_k - B\boldsymbol{u}_k) \quad \text{measured on a flight not used for learning} \tag{M2.20}$$

Measure: floor marker j through the pinhole camera, and the gyroscope:

$$\boldsymbol{q}_j = R_{bc}^\top R(\boldsymbol{\theta})^\top(\boldsymbol{\ell}_j - \boldsymbol{p}), \qquad \boldsymbol{z}_j = \begin{bmatrix} f\,q_1/q_3 + c_x \\ f\,q_2/q_3 + c_y \end{bmatrix}, \qquad \boldsymbol{z}_\omega = \boldsymbol{\omega} \tag{M2.21}$$

$$H_j = \frac{f}{q_3}\begin{bmatrix} 1 & 0 & -q_1/q_3 \\ 0 & 1 & -q_2/q_3 \end{bmatrix}\begin{bmatrix} -R_{bc}^\top R^\top & 0 & R_{bc}^\top[R^\top(\boldsymbol{\ell}_j - \boldsymbol{p})]_\times J_r(\boldsymbol{\theta}) & 0 \end{bmatrix} \tag{M2.22}$$

Correct:

$$L = \Sigma H^\top\left(H\Sigma H^\top + R_n\right)^{-1} \tag{M2.23}$$

$$\hat{\boldsymbol{x}} \leftarrow \hat{\boldsymbol{x}} + L(\boldsymbol{z} - h(\hat{\boldsymbol{x}})), \qquad \Sigma \leftarrow (I - LH)\Sigma(I - LH)^\top + LR_nL^\top \tag{M2.24}$$

Symbol	Meaning	Size	Unit
$\hat{\boldsymbol{x}}_k$, Σ_k	the estimate and how unsure it is	$\mathbb{R}^{12}$, $\mathbb{R}^{12\times12}$	as $\boldsymbol{x}_k$, its square
Q_e	how wrong the learned model is per step	$\mathbb{R}^{12\times12}$	as Σ
$\boldsymbol{\ell}_j$	known position of floor marker j (like a printed AprilTag)	$\mathbb{R}^3$	m
R_{bc}	camera axes in the body: forward, right, down = $\operatorname{diag}(1,-1,-1)$	SO(3)	–
$\boldsymbol{q}_j$	marker j seen in camera axes; q_3 is its depth	$\mathbb{R}^3$	m
f, c_x, c_y	focal length and image centre	$\mathbb{R}$	px (400, 320, 240)
$\boldsymbol{z}_j$, $\boldsymbol{z}_\omega$	pixel of marker j, gyroscope reading	$\mathbb{R}^2$, $\mathbb{R}^3$	px, rad/s
H, R_n	slope (Jacobian) of the measurements, their noise	$\mathbb{R}^{m\times12}$, $\mathbb{R}^{m\times m}$	px per unit, px^2
$J_r(\boldsymbol{\theta})$	right Jacobian of SO(3), $\approx I$ near hover	$\mathbb{R}^{3\times3}$	–
L	estimator (Kalman) gain	$\mathbb{R}^{12\times m}$	mixed

Where each line comes from

(M2.19) Predict. The learned model is a straight line already. If our error $\boldsymbol{e}_k = \boldsymbol{x}_k - \hat{\boldsymbol{x}}_k$ averages zero, then $\boldsymbol{e}_{k+1} = A\boldsymbol{e}_k + \boldsymbol{w}_k$, where $\boldsymbol{w}_k$ is the model's own error. Its average gives the first equation and its covariance gives the second, just like (M1.11) with $F = A$.

(M2.20) The noise, from the data. $\boldsymbol{w}_k = \boldsymbol{x}_{k+1} - A\boldsymbol{x}_k - B\boldsymbol{u}_k$ is exactly what the learned model cannot explain, so its covariance is Q_e. It must be measured on a *new* flight. On the training flights, least squares already made these residuals as small as possible, and the filter would become over-confident. The test flight has gusts twice as strong, and variance goes with the square, so Q_e gets a factor of 4.

(M2.21) The pinhole camera. The marker as seen from the drone is the arrow $\boldsymbol{\ell}_j - \boldsymbol{p}$. $R^\top$ turns it into the drone's axes, and $R_{bc}^\top$ into the camera's (forward, right, down). By similar triangles, something q_1 to the side at depth q_3 appears $f\,q_1/q_3$ pixels from the image centre. Dividing by the depth, and rotating by $R(\boldsymbol{\theta})$, make this a curve rather than a line. That is the only reason the filter has to be *extended*.

(M2.22) The slope, by the chain rule. pixel $\leftarrow \boldsymbol{q} \leftarrow$ state. The quotient rule gives the first matrix. Moving the drone moves the marker the other way ($-R_{bc}^\top R^\top$). Turning the drone by a small angle $\boldsymbol{d}$ changes $R^\top \boldsymbol{a}$ by $[R^\top \boldsymbol{a}]_\times J_r \boldsymbol{d}$. Velocity and rate do not move pixels (zero columns). The toy model checks this slope against a numerical one: they agree to 10^{-7} px.

(M2.23)–(M2.24) Correct. The same derivation as (M1.12)–(M1.14). The state of Model 2 is a plain list of 12 numbers (the attitude is already the 3 angles $\boldsymbol{\theta} = \mathrm{Log}R$), so correcting is ordinary addition. With up to 70 measurements at once we use the second ("Joseph") form for Σ, which keeps it symmetric and positive.

Toy example — altitude from one pixel

A camera looks down from height h at a marker $d = 1\,\mathrm{m}$ to the side, so the marker appears $u = fd/h$ pixels off-centre. Our guess: $\hat{h} = 2\,\mathrm{m}$, $\Sigma = 0.04\,\mathrm{m}^2$ ($\sigma = 20\,\mathrm{cm}$). The camera says $u = 210\,\mathrm{px}$ (noise $R_n = 4\,\mathrm{px}^2$).

$$u_{\mathrm{pred}} = \frac{400 \cdot 1}{2} = 200, \qquad H = -\frac{fd}{\hat{h}^2} = -100\,\frac{\mathrm{px}}{\mathrm{m}}, \qquad L = \frac{\Sigma H}{H^2\Sigma + R_n} = \frac{-4}{404} = -0.0099,$$

$$\hat{h} \leftarrow 2 - 0.0099 \cdot 10 = 1.901\ \mathrm{m}, \qquad \Sigma \leftarrow (1 - LH)\Sigma = 0.0004\ \mathrm{m}^2\ (\sigma = 2\ \mathrm{cm}).$$

Turning the pixel round exactly gives $400/210 = 1.905\,\mathrm{m}$. The 4 mm difference is the cost of treating the curve as a line.

Toy model: `toy_model3/M2_4_EKF_State_Estimation.mlx`, section "The closed loop"

```
[zc, ~, ids] = DroneLib.cam_measure(xd, P.p_hover, C);          % camera sees the TRUE
    drone
z = [zc + C.sigma_px*randn(size(zc)); xd(10:12) + C.sigma_gyro*randn(3,1)];
[zh, Hc] = DroneLib.cam_measure(xh, P.p_hover, C, ids);         % ... the EKF predicts
    from its estimate
Hm = [Hc; zeros(3,9), eye(3)];
[xh, Pk] = DroneLib.ekf2_update(xh, Pk, z, [zh; xh(10:12)], Hm, Rn);   % M2.23-M2.24
du = DroneLib.mpc_solve(mpc, xh);                                 % the controller flies
     on the ESTIMATE
...
[xh, Pk] = DroneLib.ekf2_predict(xh, Pk, u_app - u_trim, A, B, Q_ekf); % M2.19
```

Toy example — the whole flight on the estimate

Same start as block 10, with gusts, 1 px pixel noise and 0.01 rad/s gyro noise. The filter starts 20 cm wrong, and up to 34 markers are in view at 2 m:

controller fed by	position error	velocity error	largest speed	distance after 10 s
MPC + EKF	3.4 mm	2.8 cm/s	1.039 m/s	2.9 cm
LQR + EKF	3.5 mm	3.8 cm/s	1.239 m/s	5.9 cm
MPC on the true state	–	–	1.000 m/s	2.9 cm

The velocity is never measured, yet it is known to about 3 cm/s. The errors stay inside the $\pm 3\sigma$ band that Σ_k predicts, so the filter knows how unsure it is.

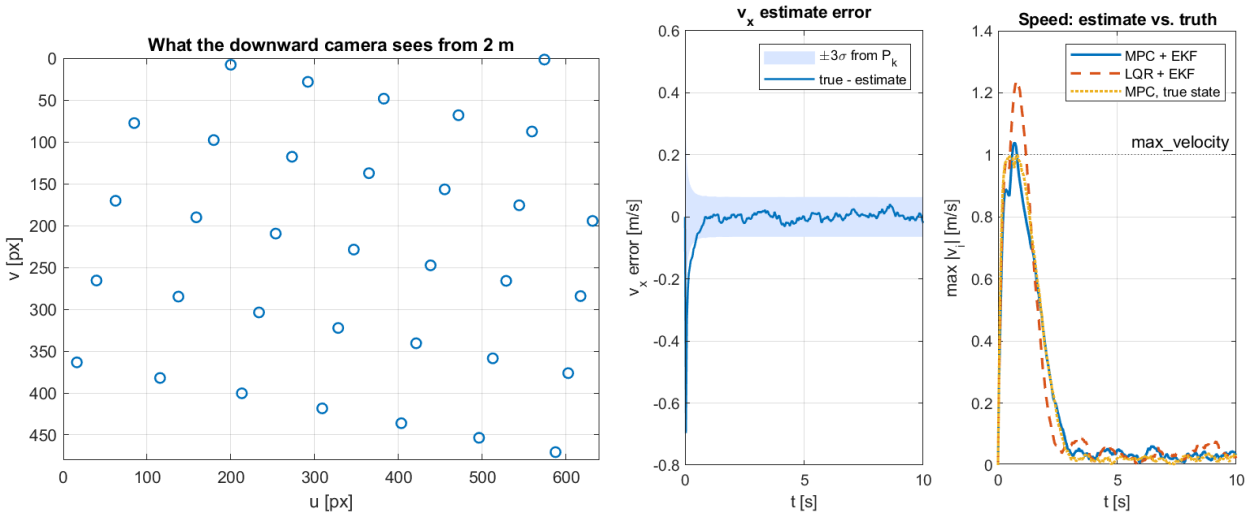


Figure 15: Left: the never-measured velocity is estimated inside its $\pm 3\sigma$ band, and flying on the estimate is close to flying on the true state. Right: what the downward camera sees from 2 m.

Limits are kept on the estimate, not on the truth

On the estimate, MPC flies at up to 1.04 m/s although its limit is 1 m/s. It keeps the limit for the state it *believes*, and gusts plus a few cm/s of estimation error make the truth slightly different. The usual cure is a small margin: plan with 0.95 m/s.

How this controls the real drone — block 11

This is the "VIO" of our title inside Model 2: the monocular camera and the IMU, fused with a model, tell the drone where it is. In the toy model the camera sees markers at known places (printed AprilTags on the floor). A full VIO system follows *natural* image features with unknown positions and adds them to the state (that is SLAM, and how the 3D map is built). The predict / measure / correct structure and the camera Jacobian (M2.22) stay the same. Today ArduPilot's EKF3 produces `/ap/v1/pose/filtered`. Block 11 is how Model 2 would run on our own camera instead.

Recap of Model 2

#	Block	In	Out	The idea in one sentence
1	Flight data	drone, random pushes	$\{\boldsymbol{x}_k\}, \{\boldsymbol{u}_k\}$	Record what the drone did and what we told it, as differences from hover.
2	DMDc	X, X^+, U	A, B	Least squares finds the linear rule that best predicts the next sample.
3	Bellman	A, B, Q, R	$V = \boldsymbol{x}^\top P \boldsymbol{x}$	Best plan = best first move + best plan from where it leads.
4	LQR	A, B, R, P	K	The bottom of a quadratic bowl is a linear law $\boldsymbol{u} = -K\boldsymbol{x}$.
5	Riccati	P_0	P_∞	Look one more step ahead each time until nothing changes.
6	DARE	A, B, Q, R	P, K	Jump straight to that limit; $\boldsymbol{x}_0^\top P \boldsymbol{x}_0$ is the total score.
7	MPC	model, limits, P_f	a problem	Plan the next second without breaking any rule.
8	QP	problem, $\boldsymbol{x}_0$	$H, \boldsymbol{f}, G, \boldsymbol{h}$, $\mathbf{U}^*$	A bowl inside a fence; one bottom, found in milliseconds.
9	Receding horizon	$\hat{\boldsymbol{x}}_k$	$\boldsymbol{u}_0^*$	Use the first move only, then plan again.
10	Apply to drone	$\boldsymbol{u}_0^*$ or $-K\hat{\boldsymbol{x}}_k$	$\Omega_1..\Omega_4$	Add hover thrust back, share among the rotors, fly.
11	EKF	$\boldsymbol{u}_k$, pixels, gyro	$\hat{\boldsymbol{x}}_k$, Σ_k	Predict with the learned model, correct with what the camera sees.

7 How Everything Fits Together

7.1 One trip around the loop, in words

Follow the drone for one 40 ms step of fig. 1, with every equation in its place.

1. **The drone moves** according to Newton and Euler, equations (1)–(4). Wind pushes it a little.
2. **The sensors measure it.** The gyroscope reads $\boldsymbol{\omega}$ 100 times a second. The camera takes a picture 25 times a second; visual odometry compares it with the previous pictures and gives a pose $(\tilde{\boldsymbol{p}}, \tilde{R})$. These same poses, together with the images, are what the 3D map is built from.
3. **The estimator (VIO) works out the state.** Between camera frames it predicts with the physics, equations (10)–(11), using the Jacobians of blocks 2–3. When a measurement arrives, it corrects, equations (12)–(14), trusting each source according to its noise. Result: $\hat{\boldsymbol{x}} = (\hat{\boldsymbol{p}}, \hat{\boldsymbol{v}}, \hat{R}, \hat{\boldsymbol{\omega}})$, published on `/ap/v1/*/filtered`.
4. **Our node checks the estimate** is fresh and finite (`state_is_valid`).
5. **Where should I be?** The waypoint generator gives $\boldsymbol{p}_d = [0, 0, 2]$, $\psi_d = 0$.
6. **How should I accelerate?** Position errors and the spring-damper law, equations (15)–(16).
7. **Which way should I lean, and how hard should I push?** Equations (17)–(19) give $\boldsymbol{F}_d$ and R_d.
8. **How should I twist?** The SO(3) attitude law, equations (20)–(22), gives $\boldsymbol{\tau}$; equation (23) gives T.

9. **What do we send?** Our node publishes all of these for monitoring and sends the velocity $\boldsymbol{v}_{\text{cmd}} = 0.8\boldsymbol{e}_p + 0.2\boldsymbol{e}_v$ (at most 1 m/s) to ArduPilot.
10. **ArduPilot finishes the job**: its own velocity, attitude and rate loops produce T and $\boldsymbol{\tau}$, and its mixer, equations (24)–(25), sets the four motor speeds. The drone moves, and we are back at step 1.

7.2 Model 1 or Model 2?

	Model 1 (physics)	**Model 2 (data)**
Where the model comes from	Newton and Euler; needs m, J, rotor data	Recorded flights (DMDc); needs nothing measured by hand
Kind of model	Nonlinear, attitude as a rotation matrix	Linear, near hover, attitude as 3 small angles
Works for	Any attitude except upside down	Small tilts (in our data, below 6.5°)
Controller	Geometric: position loop → attitude loop	LQR (a fixed gain table) or MPC (a small optimisation every step)
State estimate	Error-state EKF: physics prediction, camera pose + gyro	EKF on the learned model: camera pixels + gyro (block 11)
Limits (speed, thrust, tilt)	Only by clipping afterwards	Written into the MPC problem itself
Cost per step on the drone	A few 3×3 matrix products	LQR: one 4×12 product; MPC: a QP of about 2.6 ms
Where in our project	`so3_controller.py`, `controller_node.py`	Toy model `M2_1`–`M2_4`

The two are not rivals. Each has its own estimator built on its own model. Model 1's EKF predicts with physics, and Model 2's EKF (block 11) predicts with the learned A, B and corrects with raw camera pixels. Either can provide the state that Model 2's controllers need. Model 2's DMDc is also a check on Model 1's physics: the learned A and B agree with the Jacobians of block 2 to about 1–3%, and the data reproduces the 1.5 kg mass to within 0.4%.

7.3 Why the camera matters for control

The single most important link in the whole chain is the estimate $\hat{\boldsymbol{x}}$. Every controller in this document (SO(3), LQR, MPC, and ArduPilot's own) works only as well as the state it is given. Block 3 showed why an IMU alone fails: a tiny gyro error grows into metres of position error within seconds (t^3). The monocular camera, through VIO, is what keeps that error at a few centimetres (block 4: 2.6 cm instead of drifting away). That is the meaning of the title of our project: the drone is *controlled by* visual-inertial odometry, and the same camera poses build the 3D map.

8 Using the Toy Model

8.1 The files

File in `toy_model3/`	Blocks	What you see when you run it
`M1_1_Dynamics_Linearization.mlx`	M1: 1–3	hover check, 10° tilt, gyroscopic coupling, open-loop drift (`ode45`), Jacobians and their numerical check, linear vs. nonlinear error
`M1_2_Error_State_EKF.mlx`	M1: 4	the altitude filter by hand, then the full 12-state filter with camera and gyro, accuracy and the 3σ band
`M1_3_SO3_Control_Closed_Loop.mlx`	M1: 5–8	each control equation with the toy numbers, recovery from 60°, the motor mixer, the full closed loop, the velocity command of our node
`M2_1_Flight_Data_DMDc.mlx`	M2: 1–2	recording five flights, DMDc by hand and on the drone, comparison with physics, test on unseen data
`M2_2_LQR_Bellman_Riccati_DARE.mlx`	M2: 3–6	Bellman step, Riccati table, scalar DARE, the drone's K, cost check
`M2_3_MPC_QP_Apply_to_Drone.mlx`	M2: 7–10	two-step QP by hand, the drone MPC, MPC = LQR when no limit is active, receding horizon, nonlinear flight
`M2_4_EKF_State_Estimation.mlx`	M2: 11	altitude-from-a-pixel EKF by hand, the camera model and its Jacobian check, Q_e from a new flight, MPC and LQR flown on the EKF estimate
`DroneLib.m`	all	the helper functions: dynamics, RK4, Exp/Log, Jacobians, controller, mixer, EKF steps, DARE, QP solver
`pdf/*.mlx.pdf`	all	every live script already run and exported, with all outputs and figures

8.2 How to run them

1. Open MATLAB (tested with R2024a; no toolboxes are needed) and make `toy_model3` the current folder, because every script calls `DroneLib`.
2. Open a live script and press *Run*, or run it section by section (*Run Section*) while reading this document.
3. Model 2 files must be run in order: `M2_1` creates `dmdc_model.mat` (the learned A, B), `M2_2` adds the LQR results to it, and `M2_3` and `M2_4` use both.
4. To export a live script as PDF, change `PUBLISH = not(ready);` at the top to `PUBLISH = ready;`. The last section then writes `pdf/<name>.mlx.pdf`, exactly like the course notes.

8.3 How each live script is organised

Every file follows the same pattern as the course notes: a header, then for each block an orange title, the equations in LaTeX with their numbers $(1), (2), \ldots$ or $(M2.x)$, a short derivation, a toy example with hand-checkable numbers, and the code that reproduces those numbers. Sections that start a new topic begin with `clearvars -except PUBLISH ready`, so each part can be run on its own.

8.4 Things to try

- In the closed-loop section of `M1_3`, add `P.Kv = 2*P.Kv;` after `P = DroneLib.params();` (a stronger damper) or `P.Kp = 0.5*P.Kp;` (a weaker spring), run it again and compare `tracking_rmse_cm`.
- In `M1_2`, make the camera noise ten times larger (`sig_p = 0.5`) and see K trust the camera less.
- In `M2_1`, remove the random pushes (`exc = zeros(4,1)`) and watch `rank_Omega` drop below 16: DMDc can no longer tell the drone from the pilot.
- In `M2_3`, change the speed limit `x_max(1:3)` to 0.5 m/s and see MPC fly more slowly but still reach the target.
- In `M2_4`, set `C.sigma_px = 5` after `C = DroneLib.camera();` (a blurrier camera) and watch the position and velocity errors grow, or remove the safety factor (`Q_ekf = cov(E')`) and see the errors leave the $\pm 3\sigma$ band: the filter becomes over-confident.

A All Symbols in One Table

Symbol	Meaning	Size	Unit
$\boldsymbol{p},\ \boldsymbol{v}$	position, velocity (world, ENU)	3	m, m/s
R	attitude: body $\to$ world rotation matrix	3×3	–
$\boldsymbol{\omega}$	angular velocity in the body frame	3	rad/s
$T,\ \boldsymbol{\tau}$	total thrust, body torque	1, 3	N, N m
$\boldsymbol{u} = [T; \boldsymbol{\tau}]$	input	4	N, N m
$m,\ J,\ g_0$	mass, inertia, gravity	1, 3×3, 1	kg, kg m^2, m/s^2
$\boldsymbol{e}_3$	$[0, 0, 1]^\top$	3	–
$[\boldsymbol{a}]_\times,\ (\cdot)^\vee$	hat (cross-product matrix), vee (its inverse)	3×3, 3	–
Exp, Log	rotation vector $\leftrightarrow$ rotation matrix	–	rad
$\hat{(\cdot)},\ (\cdot)_d,\ \tilde{(\cdot)}$	estimated, desired, measured	–	–
$\delta\boldsymbol{x}$	error state $[\delta\boldsymbol{p}; \delta\boldsymbol{v}; \delta\boldsymbol{\theta}; \delta\boldsymbol{\omega}]$	12	m, m/s, rad, rad/s
$A,\ B$	Jacobians (Model 1) or learned model (Model 2)	12×12, 12×4	–
F	one-step error transition	12×12	–
P (EKF)	error covariance	12×12	squared units
$Q,\ R_n$ (EKF)	process noise, sensor noise covariance	12×12, $m \times m$	squared units
$H,\ K$ (EKF)	measurement Jacobian, Kalman gain	$m \times 12$, $12 \times m$	–
$\boldsymbol{e}_p,\ \boldsymbol{e}_v$	position, velocity error	3	m, m/s
$K_p,\ K_v$	position, velocity gains	3×3	1/s^2, 1/s
$\boldsymbol{a}_d,\ \boldsymbol{F}_d$	desired acceleration, desired force	3	m/s^2, N
$\boldsymbol{b}_{1d}, \boldsymbol{b}_{2d}, \boldsymbol{b}_{3d},\ R_d$	desired body axes, desired attitude	3, 3×3	–
ψ_d	desired yaw	1	rad
$\boldsymbol{e}_R,\ \boldsymbol{e}_\omega$	attitude error, angular-velocity error	3	rad, rad/s
$K_R,\ K_\omega$	attitude, rate gains	3×3	N m, N m s
$\Gamma,\ \boldsymbol{f},\ \boldsymbol{\Omega}$	mixer matrix, rotor thrusts, motor speeds	4×4, 4, 4	–, N, rad/s
$k_f,\ c_m$	thrust constant, yaw torque per newton	1	N s^2, m
$X,\ X^+,\ U$	snapshot matrices (DMDc)	$12 \times N$, $4 \times N$	–
$Q,\ R$ (control)	state weight, input weight	12×12, 4×4	–
P (control)	value matrix: $V(\boldsymbol{x}) = \boldsymbol{x}^\top P \boldsymbol{x}$	12×12	–
K (control)	LQR gain, $\boldsymbol{u} = -K\boldsymbol{x}$	4×12	–
$N,\ P_f$	MPC horizon, terminal weight	1, 12×12	steps, –
$S_x,\ S_u,\ H,\ \boldsymbol{f},\ G,\ \boldsymbol{h}$	QP matrices	see block 8	–
$\boldsymbol{v}_{\text{cmd}}$	velocity command sent by our node	3	m/s

B Glossary

Word	Meaning in plain words
ArduPilot SITL	the real flight-controller software, running on a PC and flying a simulated drone
Attitude	which way the drone is facing and tilted; stored as the matrix R
Covariance	a table of "how unsure" for every error and every pair of errors
DARE	the equation whose solution gives the best LQR gain for an endless flight
DMDc	learning $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$ from recorded data by least squares
EKF	Extended Kalman Filter: predict with the model, correct with sensors, weighted by trust
ENU / FLU	world axes East-North-Up / body axes Forward-Left-Up
Error state	the filter keeps track of the small error, not the full state, so the attitude stays a true rotation
Gimbal lock	the failure of roll-pitch-yaw angles at 90° pitch; avoided by using R
IMU	Inertial Measurement Unit: gyroscope and accelerometer
Innovation	what the sensor says minus what we expected it to say
Jacobian	the matrix of all "slopes" of a function of many variables
Kalman gain	how far to move the estimate towards a measurement
Linearization	replacing a curved model by its tangent (straight-line) version near a point
LQR	the best linear controller for a quadratic score: $\boldsymbol{u} = -K\boldsymbol{x}$
Lyapunov function	an "energy" that can only decrease; proves that a controller settles
MPC	re-plan the next few seconds at every step, with limits, and use only the first move
QP	quadratic program: find the bottom of a bowl inside a fence
ROS 2	software that lets programs exchange messages on named topics
SO(3)	the set of all rotation matrices
VIO	Visual-Inertial Odometry: working out motion from a camera and an IMU together

C References

[1] T. Lee, M. Leok, N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)", IEEE CDC, 2010. (Blocks 5–7 of Model 1.)

[2] J. Solà, "Quaternion kinematics for the error-state Kalman filter", arXiv:1711.02508, 2017. (Block 4.)

[3] J. L. Proctor, S. L. Brunton, J. N. Kutz, "Dynamic mode decomposition with control", SIAM J. Applied Dynamical Systems 15(1), 2016. (Model 2, block 2.)

[4] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, Vol. I. (Model 2, blocks 3–6.)

[5] J. B. Rawlings, D. Q. Mayne, M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed., 2017. (Model 2, blocks 7–9.)

[6] T. Qin, P. Li, S. Shen, "VINS-Mono: a robust and versatile monocular visual-inertial state estimator", IEEE T-RO, 2018.

[7] Sunil Kumar S and K. P. Soman, course notes *Introduction to Drones – 22AIE448*: "Taylor series" and "Nonlinear systems and EDMD", Amrita Vishwa Vidyapeetham.

[8] ArduPilot documentation: SITL with Gazebo, ROS 2 (DDS) interface. Project parameters: `drones_controller/config/controller.yaml`.